

STANDARD OPERATING PROCEDURE (SOP)

Vconstech CRM-ERP-Website Platform

1. Cover Page

Document Title	Standard Operating Procedure — CRM, ERP, and Public Website Platform
Document Type	Enterprise SOP (Technical & Operational)
Prepared For	Vconstech / Internal Engineering, Support & Operations Teams
Prepared From	Codex-generated Implementation Analysis (source of truth)
Volume	Volume 1 of 4 — Overview, Architecture, and CRM Backend Modules
Classification	Internal Use Only
Status	Draft for Review
Date Issued	Not Available (source document undated)

2. Document Information

Field	Detail
Document ID	SOP-VCT-001-VOL1
Owner	Not Available
Author	Generated from Codex Implementation Analysis
Review Cycle	Not Available
Approval Authority	Not Available
Related Volumes	Volume 2 (ERP Backend Modules), Volume 3 (Public Website & Frontend Modules), Volume 4 (Glossary, Acronyms, Revision History, Appendix)

Field	Detail
Source Document	"Implementation Analysis" — auto-generated repository analysis covering CRM Backend, CRM Frontend, ERP Backend, ERP Frontend, and Public Website
Scope of This SOP	All identified modules, workflows, APIs, data stores, business rules, and integrations described in the source Implementation Analysis. No functionality has been invented; any gap in source data is marked Not Available .

3. Table of Contents

Volume 1 (this document)

1. Cover Page
2. Document Information
3. Table of Contents
4. Project Overview
5. System Architecture Overview
6. Module Index (All Volumes)
7. CRM Backend Modules
 - 7.1 Auth Module
 - 7.2 Lead Module
 - 7.3 Lead Work History Module
 - 7.4 Customer Module
 - 7.5 Subscription Conversion Module
 - 7.6 Plan Module
 - 7.7 Task Module
 - 7.8 Ticket Module
 - 7.9 Notification Module
 - 7.10 Department Module
 - 7.11 Dashboard Module
 - 7.12 CRM Integration Module
 - 7.13 CRM Scheduler Modules (Follow-Up Reminder & Subscription Expiry)

Volume 2 — ERP Backend Modules (Auth, Invitation, Registration, Subscription Sync, Webhook/Payment Notification, Project, Material, Project Material, Material Request, Usage Log, Notification, Engineer, User/Profile, Super Admin, Financial, Contract, Labour, Billing, Daily Progress, Report)

Volume 3 — Public Website Modules (Pricing, Payment Result, Invitation Registration, Contact/Floating Buttons) and Frontend Modules (CRM Frontend, ERP Frontend)

Volume 4 — Glossary, Acronyms, Revision History, Change Log, Appendix

4. Project Overview

The platform consists of **five connected applications** operating as a split CRM / ERP / public-website stack:

#	Application	Path	Technology	Purpose
1	CRM Backend	vsoft-spilit-backup/backend	Express.js + PostgreSQL (pg)	Manages CRM employees, leads, subscription plans, tasks, tickets, notifications, CRM-ERP integrations, reminders, and paid subscriptions
2	CRM Frontend	vsoft-spilit-backup/Vconstech	React / Vite	CRM dashboard: login, lead management, subscription plans, member management, department task/ticket pages, invitation management
3	ERP Backend	Server_codes/Vconstech_backend	Express.js + Prisma + PostgreSQL	Manages ERP users, comparison materials, material requests, billing, contracts, labour, financials, invitations, registrations, supplier management, Razorpay webhooks, payment gateway
4	ERP Frontend	Server_codes/Vconstech_frontend	React / Vite	ERP admin dashboards, project/material/financial/contract views, engineer login/worksheet management
5	Public Website	Vconstech-site-master	React / Vite	Marketing pages, contact/feedback, registration, pricing page, Razorpay checkout, payment result page, ERP synchronization, ERP purchase

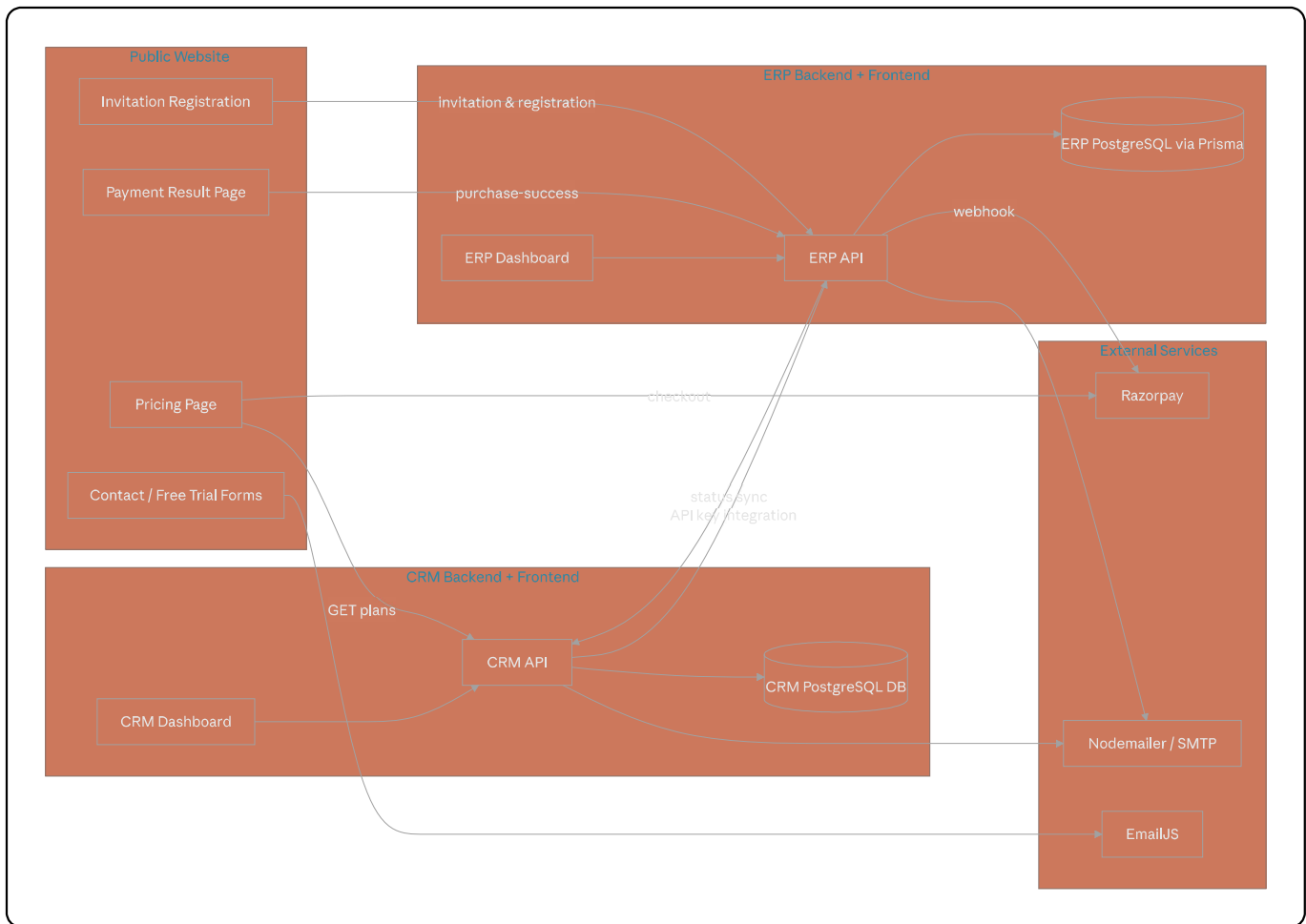
Static image/video assets and upload folders serve as content storage and do not define business behavior, except where upload controllers store or serve file paths.

The system supports the full sales-to-operations lifecycle:

1. Lead capture and conversion in the CRM.
 2. Customer invitation and trial account provisioning in the ERP.
 3. Public-website subscription purchase (trial upgrade or direct purchase) via Razorpay.
 4. Ongoing construction-project operations (projects, materials, labour, billing, contracts, financials) inside the ERP.
 5. Subscription renewal reminders and expiry handling, with CRM/ERP account-status synchronization.
-

5. System Architecture Overview

- CRM owns lead, customer, plan, follow-up, task, ticket, and sales lifecycle data.
- ERP owns ERP account login, construction project execution data, company-scoped operational data, trial state, subscription activation state, and ERP account status.
- Public website displays CRM-managed pricing plans and forwards pricing purchases to ERP subscription sync.
- CRM and ERP communicate through API-key-protected integration endpoints.
- Razorpay checkout runs from the public website; the ERP backend contains a Razorpay webhook route for server-side payment notification handling.
- Email is sent from the CRM backend via Nodemailer utilities, from the ERP backend via `src/utils/mailer.js`, and from the public website via EmailJS for contact/free-trial forms.



5.2 System Data Stores

5.2.1 CRM Database (PostgreSQL via `pg`)

Configured in `vsoft-spilit-backup/backend/config/database.js`.

Table	Purpose
<code>team_members</code>	CRM employees, credentials, departments, reset tokens
<code>leads</code>	Sales leads: status, contact data, plan, requirements, assignment, follow-up date/time, reminder flags, conversion marker
<code>customers</code>	CRM customers: lead reference, contact data, plan, payment/subscription state, ERP references, trial dates, subscription dates, reminder fields
<code>subscription_history</code>	Audit history of customer subscription actions: <code>NEW</code> , <code>PLAN_CHANGED</code> , <code>ACTIVATED</code> , <code>DEACTIVATED</code> , <code>RENEWED</code> , <code>EXPIRED</code>
<code>plans</code>	CRM subscription plan records
<code>plan_features</code>	Plan feature rows linked to <code>plans.id</code>
<code>tasks</code>	Assigned CRM tasks with department/status/updated metadata
<code>task_updates</code>	Employee work notes for tasks
<code>tickets</code>	CRM support/workspace tickets: ticket number, type, state, urgency, department, assignee, caller, due date, description
<code>ticket_number_counters</code>	Per-ticket-type counters for incident/request ticket numbers
<code>ticket_history</code>	Ticket status/worknote/follow-up history
<code>notifications</code>	CRM employee notifications
<code>crm_erp_customer_mappings</code>	CRM-to-ERP invitation/customer mapping and idempotency state
<code>crm_erp_status_events</code>	Idempotent CRM events for ERP status sync, renewal reminders, subscription expiry processing

`database/ensureSchema.js` runs at CRM startup and applies non-destructive `ALTER TABLE`, `CREATE TABLE IF NOT EXISTS`, and `CREATE INDEX IF NOT EXISTS` statements.

5.2.2 ERP Database (Prisma + PostgreSQL)

Defined in `Server_codes/Vconstech_backend/prisma/schema.prisma`. Primary models: `User`, `Company`, `Client`, `CrmInvitation`, `CustomerSubscription`, `Project`, `Engineer`,

ProjectAssignment, Material, ProjectMaterial, MaterialUsage, MaterialRequest, Notification, Contract, FinancialTransaction, ProjectExpense, File, Labour, LabourPayment, daily_progress_updates, Bill, BillItem, PaymentNotification. Full detail is provided in the respective module sections of Volume 2.

5.3 Request/Response Pattern Standards

Application	Pattern
CRM	Success object { success: true, ... }; raw array for list endpoints (e.g. GET /api/leads, GET /api/customers, GET /api/plans); validation errors { success: false, message: "Please fix the highlighted fields.", errors: { ... } }; conflict duplicates with field-specific flags; server errors { success: false, message: err.message } or { error: err.message }
ERP	{ success: true, data } for subscription sync/integration endpoints; { message, token, user } for auth; { success: false, error, details } for service errors; module-specific shapes such as { projects, count }, { material }, { bills }, { notifications }

5.4 Cross-Cutting Environment Variables

Full environment variable listings for each application are reproduced in the Configuration/Environment Variables sections of the relevant module. A consolidated table appears in Volume 4, Appendix A.

6. Module Index (All Volumes)

#	Module	Application	Volume
1	Auth Module	CRM Backend	1
2	Lead Module	CRM Backend	1
3	Lead Work History Module	CRM Backend	1
4	Customer Module	CRM Backend	1

#	Module	Application	Volume
5	Subscription Conversion Module	CRM Backend	1
6	Plan Module	CRM Backend	1
7	Task Module	CRM Backend	1
8	Ticket Module	CRM Backend	1
9	Notification Module	CRM Backend	1
10	Department Module	CRM Backend	1
11	Dashboard Module	CRM Backend	1
12	CRM Integration Module	CRM Backend	1
13	CRM Scheduler Modules	CRM Backend	1
14	ERP Auth Module	ERP Backend	2
15	ERP Invitation Module	ERP Backend	2
16	ERP Registration Module	ERP Backend	2
17	ERP Subscription Sync Module	ERP Backend	2
18	ERP Webhook & Payment Notification Module	ERP Backend	2

#	Module	Application	Volume
19	ERP Project Module	ERP Backend	2
20	ERP Material Module	ERP Backend	2
21	ERP Project Material Module	ERP Backend	2
22	ERP Material Request Module	ERP Backend	2
23	ERP Usage Log Module	ERP Backend	2
24	ERP Notification Module	ERP Backend	2
25	ERP Engineer Module	ERP Backend	2
26	ERP User/Profile Module	ERP Backend	2
27	ERP Super Admin Module	ERP Backend	2
28	ERP Financial Module	ERP Backend	2
29	ERP Contract Module	ERP Backend	2
30	ERP Labour Module	ERP Backend	2
31	ERP Billing Module	ERP Backend	2
32	ERP Daily Progress Module	ERP Backend	2

#	Module	Application	Volume
33	ERP Report Module	ERP Backend	2
34	Pricing Module	Public Website	3
35	Payment Result Module	Public Website	3
36	Invitation Registration Module	Public Website	3
37	Contact & Floating Buttons Module	Public Website	3
38	CRM Frontend Modules (Routing/Layout, Lead UI, Customer UI, Subscription Plan UI, Team Member UI, Ticket/Workspace UI, Department UI)	CRM Frontend	3
39	ERP Frontend Modules (Routing, Admin Pages, Employee Pages, Super Admin Pages, API Clients)	ERP Frontend	3

7. CRM Backend Modules

7.1 Auth Module

Module Name

CRM Auth Module

Purpose

Authenticates CRM employees by email, password, and department, and manages forgot/reset/change password workflows.

Scope

Covers CRM employee login, logout (client-side session clear), password reset request, password reset confirmation, and password change. Does not cover ERP or super admin authentication (see Volume 2).

Business Overview

CRM staff must select their department at login in addition to credentials, ensuring a person cannot log into a department workspace they are not assigned to. No JWT is issued; the frontend persists the authenticated employee object in `localStorage`.

User Roles

Sales, Support, Technical, Marketing, and Team/admin-style dashboard users (roles/departments stored on `team_members`).

Functional Requirements

- FR-1: Employee must authenticate with email, password, and department.
- FR-2: System must support password reset via emailed token.
- FR-3: System must support authenticated in-app password change.

Preconditions

- Employee record must already exist in `team_members`.
- SMTP/Email credentials (`EMAIL_USER`, `EMAIL_PASS`) must be configured for reset emails to be delivered.

Workflow

Login:

1. CRM login screen posts `email`, `password`, and `department` to `POST /api/auth/login`.
2. Backend fetches `team_members` by email.
3. Department is compared case-insensitively with the selected department.
4. Password is compared using `bcryptjs.compare`.
5. Response returns the employee record without password/reset token fields.
6. CRM frontend stores the employee object in `localStorage`.
7. `ProtectedRoute` in the CRM frontend checks `localStorage.getItem("employee")` on every protected navigation.

Password Reset:

1. `POST /api/auth/forgot-password` locates a team member by email.
2. A 32-byte hex token is generated using Node `crypto`.
3. Token and a 15-minute expiry are stored on `team_members`.
4. `sendPasswordResetEmail` sends the reset link.

5. `POST /api/auth/reset-password/:token` verifies token and expiry, hashes the new password with bcrypt, and clears reset token fields.
6. `POST /api/auth/change-password` requires employee ID, old password, and a new password of at least 8 characters.

System Execution Flow

Request → Route (`routes/authRoutes.js`) → Controller (`controllers/authController.js`) → PostgreSQL (`team_members`) → bcrypt validation → Response.

Data Flow

Input: email, password, department (login); email (forgot password); token + new password (reset); employeeId + old/new password (change). Output: sanitized employee object (login); generic confirmation (forgot password); success flag (reset/change).

API Documentation

Method	Endpoint	Description	Auth
POST	<code>/api/auth/login</code>	Employee login	None
POST	<code>/api/auth/forgot-password</code>	Request reset token	None
POST	<code>/api/auth/reset-password/:token</code>	Reset password via token	Token in URL
POST	<code>/api/auth/change-password</code>	Change password while authenticated	Employee session

Database Documentation

Reads and updates `team_members` (email, password hash, department, reset token, reset token expiry).

Business Rules

- Login is rejected if department does not match (case-insensitive).
- Reset tokens expire after 15 minutes.
- No JWT issuance; session state is client-side only.

Validation Rules

- Login requires email, password, department.
- Change password requires new password length ≥ 8 characters.

- Reset token must exist and not be expired.

Error Handling

- 400 — missing data.
- 401 — invalid credentials or department mismatch.
- 404 — unknown reset email.
- 500 — unexpected errors.

Notifications

Password reset email sent via Nodemailer (`config/email.js`, `utils/emailUtil.js`). No in-app notification generated for auth events.

Security

- Passwords hashed with `bcryptjs`.
- Reset tokens generated using `crypto.randomBytes(32)` (cryptographically random) with 15-minute expiry.
- **Gap:** CRM frontend protected routes rely on `localStorage` employee state rather than a signed token (documented security gap; see Volume 4 Appendix).

Configuration

Email transport configured via `config/email.js`.

Environment Variables

`EMAIL_USER`, `EMAIL_PASS`.

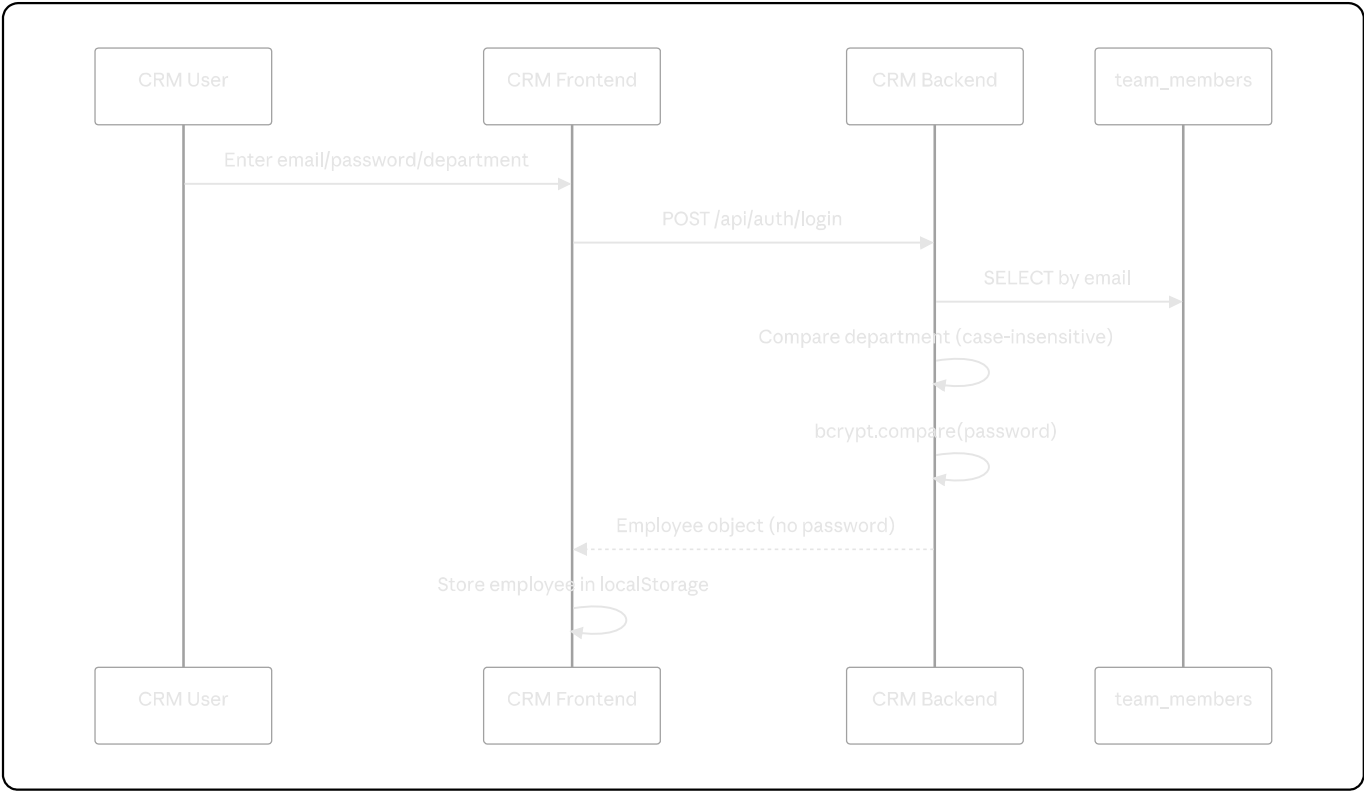
Dependencies

Express, `bcryptjs`, Nodemailer, PostgreSQL (`pg`).

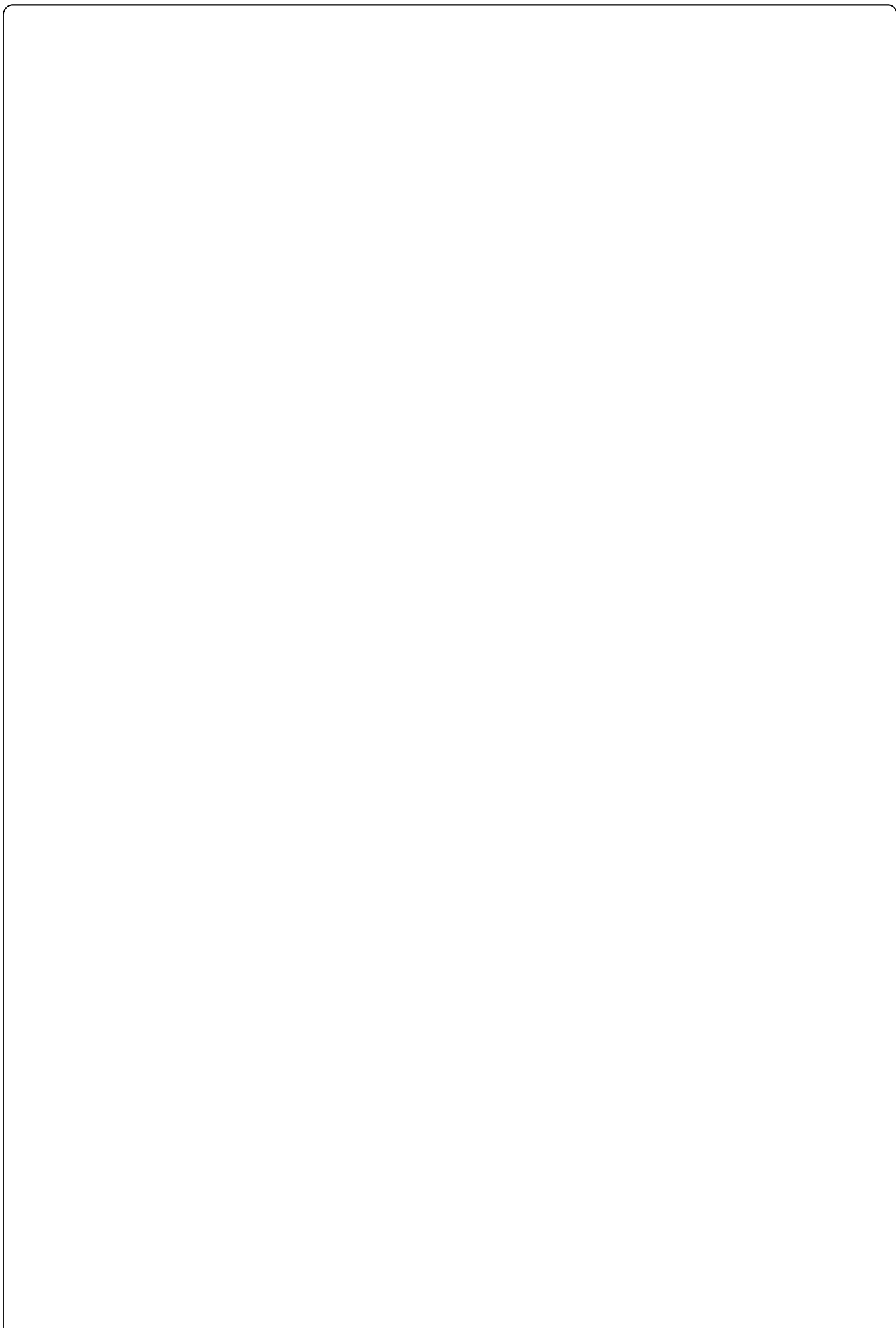
Important Files

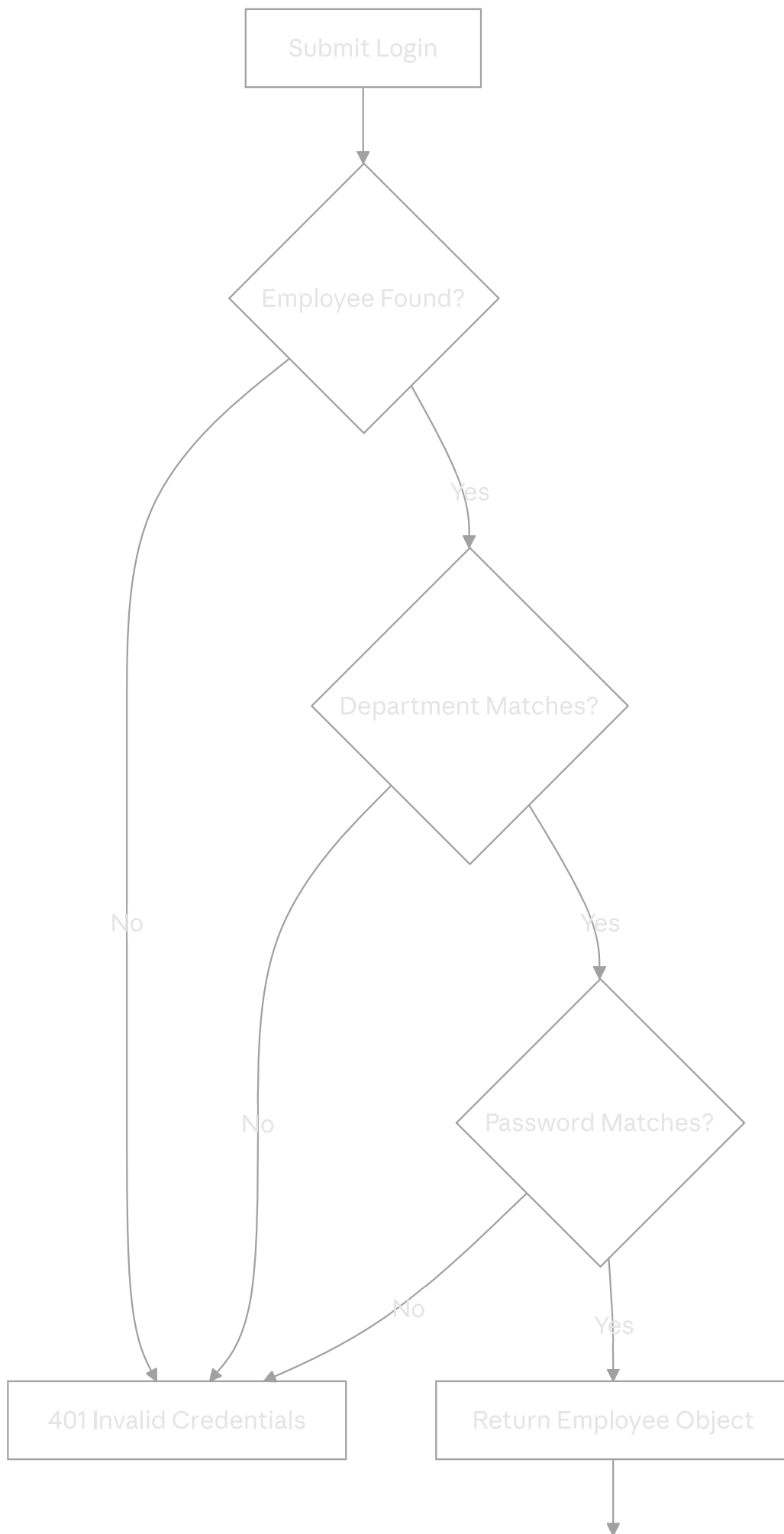
`controllers/authController.js`, `routes/authRoutes.js`,
`Vconstech/src/components/Authentication/Login.jsx`, `ForgotPassword.jsx`,
`ResetPassword.jsx`.

Mermaid Sequence Diagram



Mermaid Flowchart





Audit Logging

Not Available (no audit log table described in source).

Performance Considerations

Not Available.

Troubleshooting

Not Available beyond standard 401/404/500 handling described above.

Expected Outcome

Authenticated CRM employee session is established client-side; password lifecycle (forgot/reset/change) functions end-to-end.

Postconditions

Employee object stored in `localStorage`; reset token cleared after successful reset.

Assumptions

Department names in the frontend match those stored in `team_members` (case-insensitive).

Limitations

No server-issued session token; no server-side session invalidation/logout enforcement described.

Best Practices

Rotate `EMAIL_USER`/`EMAIL_PASS` credentials periodically; monitor for repeated failed login attempts (no rate limiting described in source).

Future Improvements

Replace CRM localStorage-only auth with signed tokens and backend middleware (per source "Future Improvements").

References

Source Implementation Analysis §"CRM Modules → Auth Module", §"Authentication Flow → CRM Authentication".

7.2 Lead Module

Module Name

CRM Lead Module

Purpose

Creates, lists, updates, deletes, and converts sales leads; supports follow-up reminder metadata.

Scope

Lead CRUD, duplicate detection, conversion trigger to customer. Excludes work-history/update sub-records (see §7.3) and the conversion transaction itself (see §7.5).

Business Overview

Leads represent prospective customers tracked through a sales pipeline. Leads may be converted into paying customers, at which point they become immutable in the Lead module.

User Roles

CRM Sales department staff; Team/admin dashboard users.

Functional Requirements

- FR-1: Create, read, update, delete leads.
- FR-2: Prevent duplicate leads by normalized email/phone.
- FR-3: Prevent edits to converted leads.
- FR-4: Support follow-up date/time and reminder flags.
- FR-5: Trigger conversion to customer.

Preconditions

Lead payload must pass validation (see below); requesting user must be authenticated CRM employee (session-based).

Workflow

1. Sales user submits lead via `AddLeadForm.jsx`.
2. `controllers/leadController.js` validates payload, normalizes phone/email/text.
3. Duplicate lookup performed by lowercase email or normalized phone.
4. Lead persisted to `leads` table.

5. Lead can be updated/viewed/deleted while unconverted.
6. Conversion is delegated to `subscriptionController.convertLeadToCustomer` (see §7.5).

System Execution Flow

Frontend form → `routes/leadRoutes.js` → `controllers/leadController.js` → PostgreSQL `leads` table → Response → Frontend list/board refresh.

Data Flow

Input: name, phone, email, requirements, plan, status, channel, date, assignment. Output: lead record; validation error map; duplicate conflict flags.

API Documentation

Method	Endpoint (under <code>/api/leads</code>)	Description
Various (CRUD)	<code>/api/leads</code>	Lead CRUD
Various	<code>/api/leads/:id/*</code>	Work history, updates
POST	<code>/api/leads/:id/convert-to-customer</code>	Converts lead, creates/reuses linked customer, writes subscription history, marks lead, triggers ERP invitation

Exact HTTP verbs/sub-paths beyond the above: Not Available (source lists route file only, not full verb table).

Database Documentation

Table: `leads`. Duplicate lookup by lowercase email or normalized phone. Conversion checks `customers.lead_id`.

Business Rules

- Lead date cannot be in the future.
- Plan is required unless status is `new`.
- Converted leads cannot be edited.
- Duplicate email/phone is rejected.

Validation Rules

- Name minimum 3 characters.
- Indian mobile number pattern `[6-9]\d{9}`.

- Valid email regex.
- Requirements minimum 10 characters when provided.

Error Handling

- 400 — validation errors.
- 409 — duplicate or converted-lead update attempt.
- 404 — not found.
- 500 — unexpected errors.

Notifications

Not directly generated by the Lead module; conversion path triggers CRM Integration Module notifications (see §7.12).

Security

Standard CRM session-based access; no field-level ACL details provided (Not Available).

Configuration

Not Available beyond general CRM backend configuration.

Environment Variables

None specific to this module (uses shared `DATABASE_URL`).

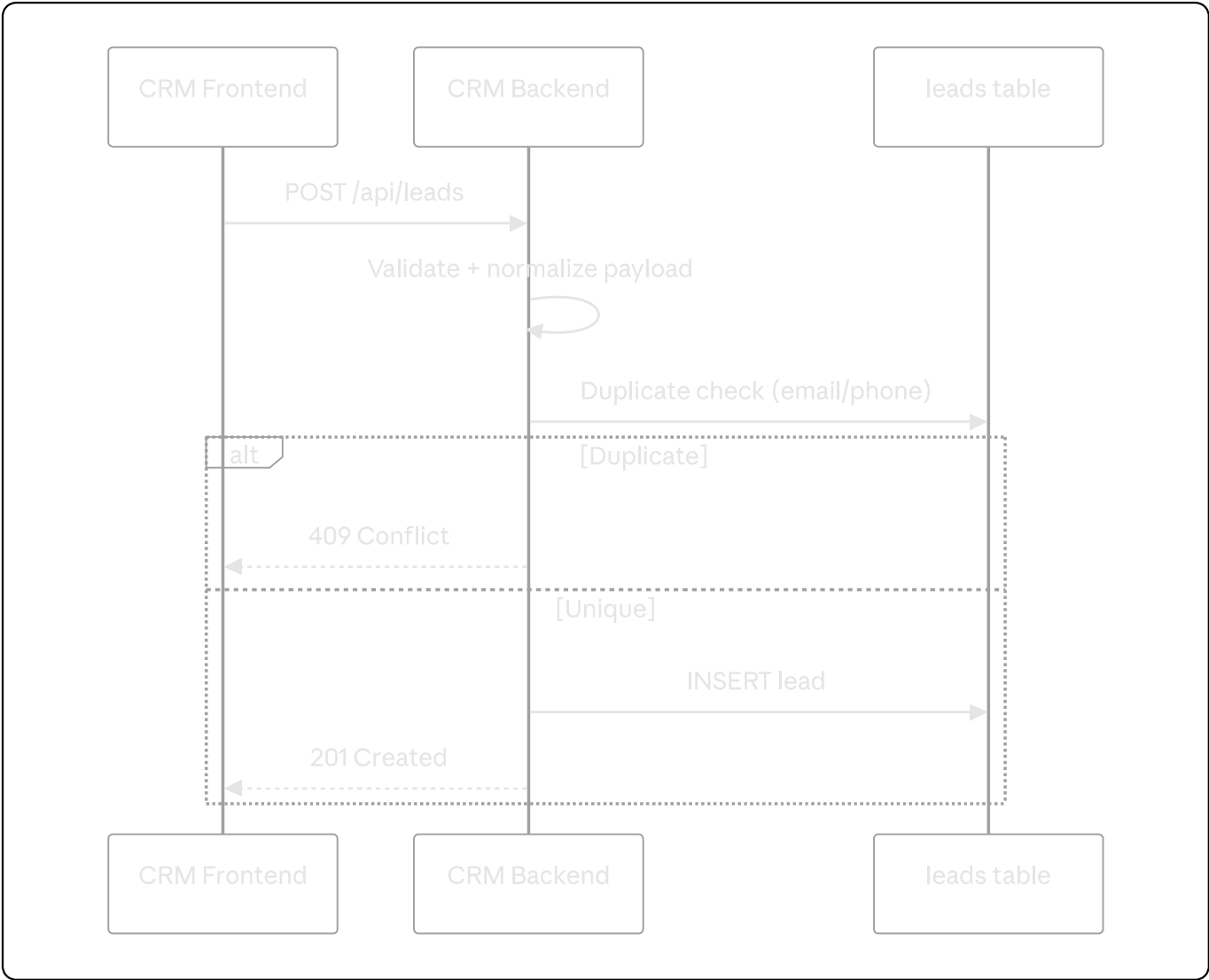
Dependencies

Express, PostgreSQL (`pg`).

Important Files

`controllers/leadController.js`, `routes/leadRoutes.js`, CRM Frontend: `AddleadForm.jsx`, `EditLeadPage.jsx`, `LeadCard.jsx`, `LeadPipelineColumn.jsx`, `LeadPipelineSection.jsx`, `LeadFilterBar.jsx`, `LeadStatsCard.jsx`, `LeadStatsSection.jsx`, `LeadsViewPage.jsx`, `Pages/LeadSales.jsx`.

Mermaid Sequence Diagram



Audit Logging

Not Available (no dedicated lead audit table; `subscription_history` covers post-conversion state only).

Performance Considerations

Not Available.

Troubleshooting

Not Available beyond standard error codes above.

Expected Outcome

Sales leads are captured, tracked through pipeline stages, and reliably converted into customers without duplication.

Postconditions

Lead marked `is_customer` upon conversion (set by Subscription Conversion Module).

Assumptions

Frontend enforces the same validation rules as backend prior to submission (not confirmed in source; assumption only).

Limitations

No bulk import/export functionality described.

Best Practices

Enforce phone/email normalization consistently between frontend and backend to avoid false duplicate rejections.

Future Improvements

Add automated tests for CRM conversion path (per source "Future Improvements").

References

Source Implementation Analysis §"CRM Modules → Lead Module".

7.3 Lead Work History Module

Module Name

CRM Lead Work History Module

Purpose

Records work history and updates for leads.

Scope

Create/get work history entries; create/get lead updates. Sub-feature of the Lead Module.

Business Overview

Provides a chronological audit trail of sales activity performed against a lead, supporting the CRM workspace view.

User Roles

CRM Sales staff.

Functional Requirements

- FR-1: Create work history entry for a lead.
- FR-2: Retrieve work history for a lead.
- FR-3: Create/get lead updates.

Preconditions

Associated lead must exist.

Workflow

1. Exposed through `routes/leadRoutes.js` using `leadWorkHistoryController`.
2. Dedicated controller methods handle create/get for work history and updates.

System Execution Flow

Frontend workspace action → `leadRoutes.js` → `leadWorkHistoryController` → lead work history/update tables → Response.

Data Flow

Input: lead ID, note/update text, employee ID. Output: history/update record(s).

API Documentation

Not Available (exact endpoint paths not enumerated in source beyond routing through `leadRoutes.js`).

Database Documentation

Reads/writes lead work history/update tables used by the CRM workspace. Exact table names: Not Available (not explicitly named in source beyond "lead work history/update tables").

Business Rules

Not Available beyond association with a valid lead.

Validation Rules

Not Available.

Error Handling

Not Available (assume standard CRM error pattern: 400/404/500).

Notifications

Not Available.

Security

Standard CRM session-based access.

Configuration

Not Available.

Environment Variables

None specific.

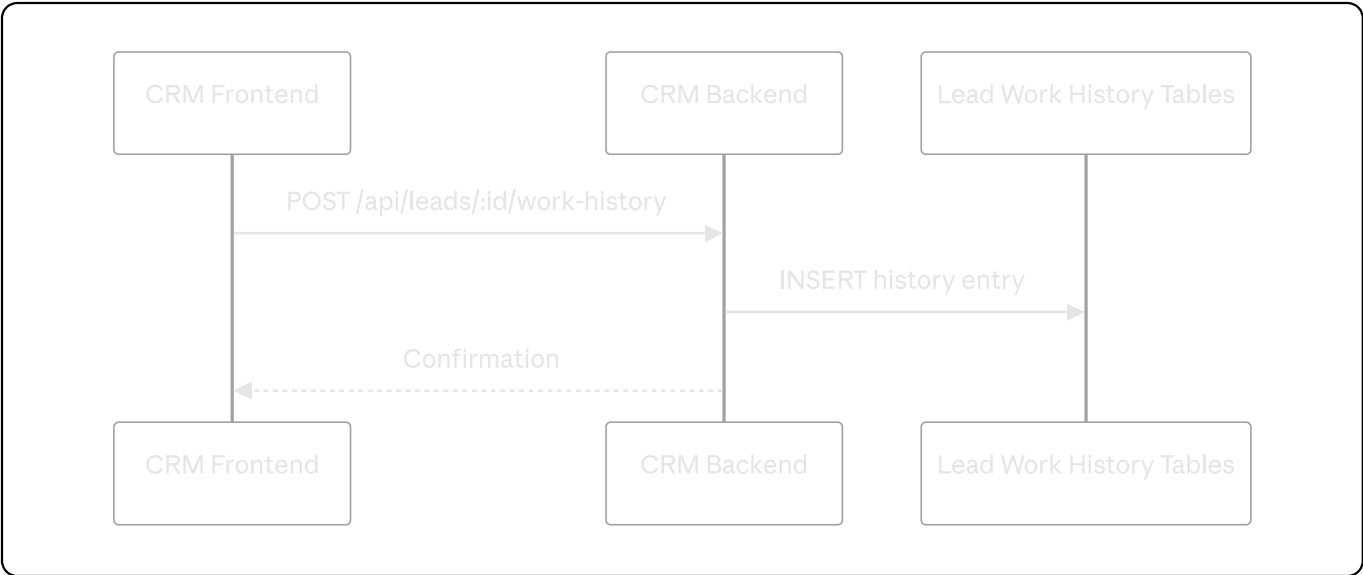
Dependencies

Express, PostgreSQL (`pg`).

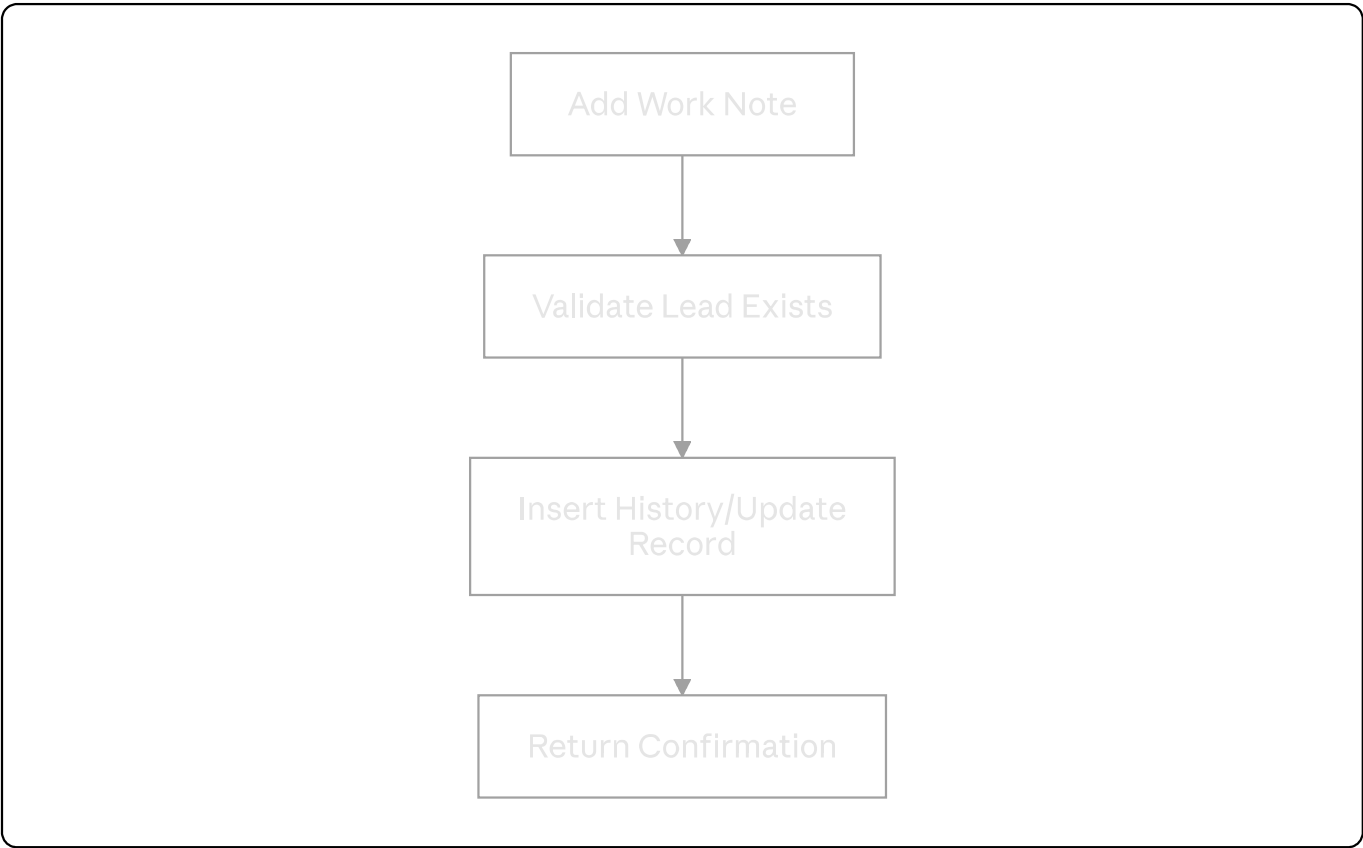
Important Files

`routes/leadRoutes.js` (work history/update routes), `leadWorkHistoryController` (controller name as referenced in source).

Mermaid Sequence Diagram



Mermaid Flowchart



Audit Logging

This module functions as the audit trail for lead activity itself.

Performance Considerations

Not Available.

Troubleshooting

Not Available.

Expected Outcome

Complete activity trail per lead available to sales staff.

Postconditions

New history/update row persisted against the lead.

Assumptions

Work history entries are immutable once created (typical audit-trail behavior; not explicitly confirmed).

Limitations

Not Available.

Best Practices

Encourage timely logging of work notes to maintain pipeline visibility.

Future Improvements

Not Available (no module-specific improvement listed in source).

References

Source Implementation Analysis §"CRM Modules → Lead Work History Module".

7.4 Customer Module

Module Name

CRM Customer Module

Purpose

Manages CRM customers, subscription dates, renewals, reminders, status, and duplicate prevention.

Scope

Customer CRUD, plan change recalculation, manual status changes, renewal processing, reminder emails.

Business Overview

The Customer entity is the CRM record of truth for a paying (or trial) client relationship, mirrored against ERP account state through integration fields.

User Roles

CRM Sales/Support staff; Team/admin dashboard users.

Functional Requirements

- FR-1: Create/update/view/delete customers.
- FR-2: Prevent duplicate customers by normalized email/phone.
- FR-3: Recalculate subscription dates on plan change.
- FR-4: Record subscription history events.
- FR-5: Send manual subscription reminder emails.

Preconditions

Plan must exist in `plans` table for date calculation; customer either created directly or via lead conversion.

Workflow

1. `controllers/customerController.js` receives request, delegates validation/duplicate detection/creation to `services/customerService.js`.
2. `services/subscriptionDateService.js` calculates plan duration dates and resolves plan data.
3. On plan change: dates recalculated, `PLAN_CHANGED` history written.
4. On manual status toggle: `ACTIVATED` or `DEACTIVATED` history written.
5. On renewal: `RENEWED` history written.
6. `sendSubscriptionReminderEmail` sends manual reminder emails on demand.

System Execution Flow

CRM Frontend (`Customerform.jsx`, `CustomerTable.jsx`) → `routes/customerRoutes.js` → `controllers/customerController.js` → `services/customerService.js` / `services/subscriptionDateService.js` → PostgreSQL (`customers`, `subscription_history`, `plans`) → Response.

Data Flow

Input: customer contact details, plan, payment/subscription state, ERP references. Output: customer record; subscription history entries; email dispatch confirmation.

API Documentation

Base Path	Route File	Purpose
<code>/api/customers</code>	<code>routes/customerRoutes.js</code>	Customer CRUD, renewal, reminders, subscription history

Database Documentation

- CRUD on `customers`.
- Writes `subscription_history`.
- Reads `plans` for price/duration mapping.

Business Rules

- Customer duplicates checked by normalized email and phone.
- Plan changes recalculate subscription dates and create `PLAN_CHANGED` history.
- Manual active/inactive status updates write `ACTIVATED`/`DEACTIVATED` history.
- Renewals write `RENEWED` history.

Validation Rules

Centralized in `customerService.validateCustomerPayload`; normalizes email, phone, and plan names.

Error Handling

- 400 — validation errors.
- 409 — duplicate customers.
- 404 — missing customer.
- 500 — unexpected errors.

Notifications

Manual subscription reminder email via `sendSubscriptionReminderEmail` (Nodemailer). Automated reminders are handled by the CRM Scheduler Module (§7.13).

Security

Standard CRM session-based access.

Configuration

Uses shared CRM email configuration (`config/email.js`).

Environment Variables

`EMAIL_USER`, `EMAIL_PASS` (for reminder emails); `DATABASE_URL`.

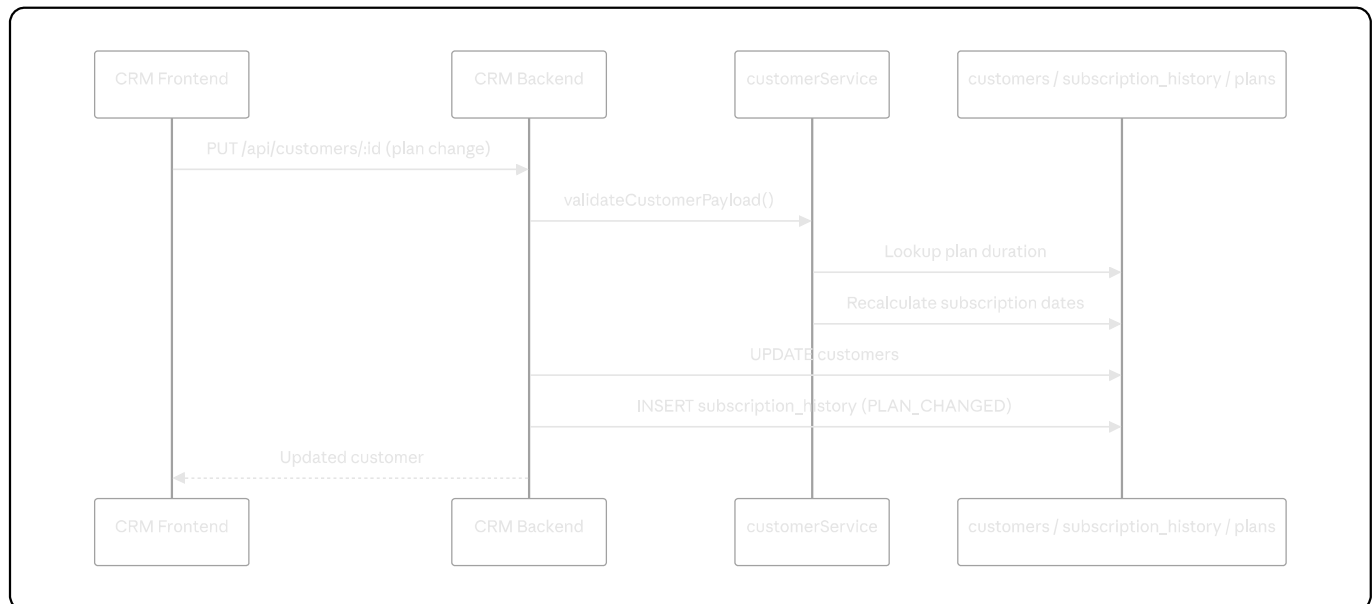
Dependencies

Express, PostgreSQL (`pg`), Nodemailer.

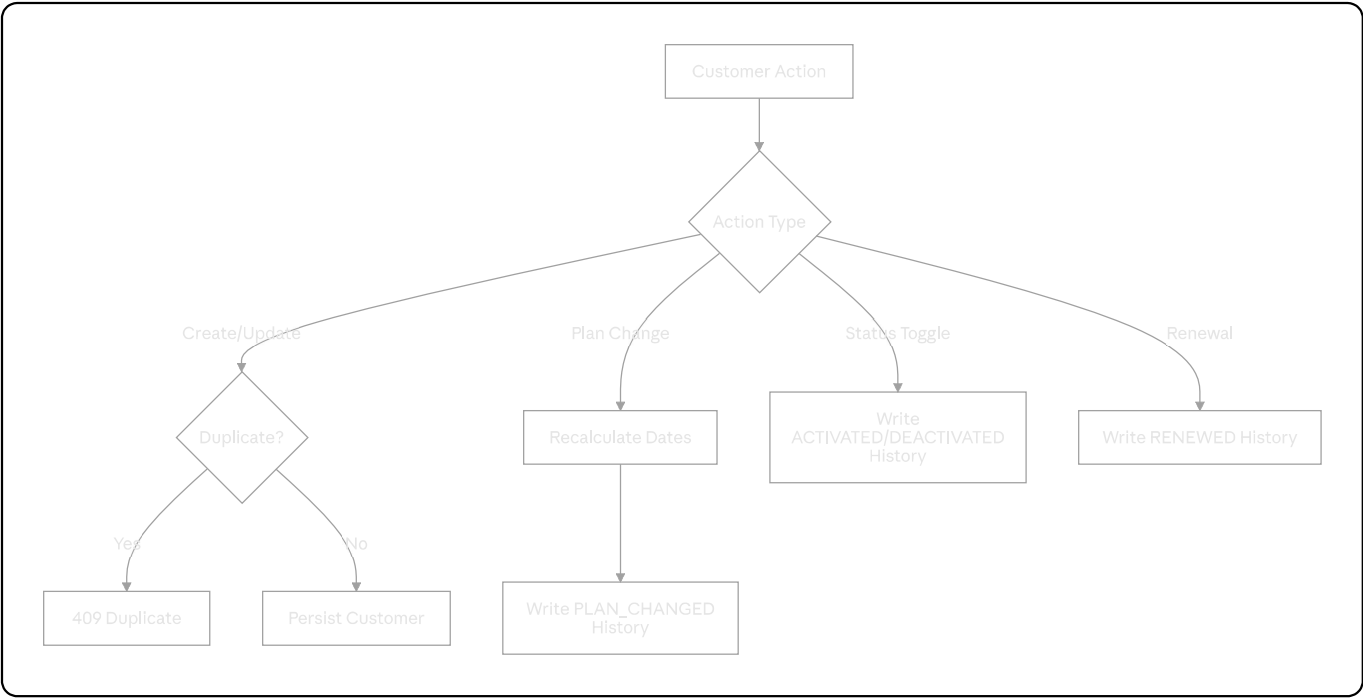
Important Files

`controllers/customerController.js`, `services/customerService.js`,
`services/subscriptionDateService.js`, CRM Frontend: `Customerform.jsx`,
`CustomerTable.jsx`, `CustomerStats.jsx`, `CustomerToolbar.jsx`, `CustomerViewPanel.jsx`,
`CustomerActionMenu.jsx`, `CustomerMobileCards.jsx`, `CustomerSearch.jsx`,
`Pages/CustomerPage.jsx`.

Mermaid Sequence Diagram



Mermaid Flowchart



Audit Logging

`subscription_history` table serves as the audit log for all customer subscription state transitions.

Performance Considerations

Not Available.

Troubleshooting

Not Available beyond standard error codes above.

Expected Outcome

Accurate, duplicate-free customer records with a complete subscription history audit trail.

Postconditions

`subscription_history` updated for every material subscription state change.

Assumptions

Plan duration data in `plans` is kept current to support accurate date recalculation.

Limitations

Not Available.

Best Practices

Always route plan changes through the Customer Module (not direct DB edits) to preserve the `subscription_history` audit trail.

Future Improvements

Standardize API response shapes across modules (per source "Future Improvements").

References

Source Implementation Analysis §"CRM Modules → Customer Module".

7.5 Subscription Conversion Module

Module Name

CRM Subscription Conversion Module

Purpose

Converts won CRM leads into CRM customers and triggers the ERP invitation flow.

Scope

Lead-to-customer transactional conversion; ERP invitation trigger. Excludes ERP-side invitation processing (Volume 2 §ERP Invitation Module).

Business Overview

This module is the transactional boundary between sales (CRM) and provisioning (ERP). It guarantees a lead is converted exactly once and safely handles partial failure (CRM success / ERP failure).

User Roles

CRM Sales staff (initiates conversion via Lead Module UI).

Functional Requirements

- FR-1: Convert a lead to a customer within a single database transaction.
- FR-2: Reuse an existing customer if the lead was already converted.
- FR-3: Trigger ERP invitation after CRM commit.
- FR-4: Prevent duplicate customer creation for the same lead.

Lead must exist and be in a convertible state (not already converted without a linked customer causing conflict).

Workflow

1. `controllers/subscriptionController.js` opens a PostgreSQL transaction.
2. Lead row is locked using `SELECT ... FOR UPDATE`.
3. If lead already converted with a linked customer, existing customer is reused.
4. If lead marked converted but no customer exists, a 409 is returned.
5. New customer created (if not reused); `subscription_history` `NEW` entry written.
6. `leads.is_customer` updated.
7. Plan data resolved through `getPlanForSubscription`.
8. Transaction commits.
9. `triggerLeadWonInvitation` is called after commit (CRM Integration Module, §7.12).

System Execution Flow

Lead Module (`convert-to-customer` action) → `subscriptionController.js` → PostgreSQL transaction (`leads` lock → `customers` insert → `subscription_history` insert → `leads` update) → commit → `leadWonIntegrationService` (post-commit, async trigger).

Data Flow

Input: lead ID. Output: customer record (new or reused); subscription history `NEW` entry; ERP invitation trigger payload (see §7.12 Data Flow).

API Documentation

Method	Endpoint	Description
POST	<code>/api/leads/:id/convert-to-customer</code>	Converts a lead, creates or reuses the linked customer, writes subscription history, marks the lead, and triggers ERP invitation

Database Documentation

- Reads `leads` (with row lock).
- Inserts `customers`.
- Inserts `subscription_history`.
- Updates `leads.is_customer`.

- Reads `plans` via `getPlanForSubscription`.

Business Rules

- No duplicate customer for the same lead.
- If lead is already marked converted but no customer exists, returns 409 (data-integrity guard).
- ERP invitation failure returns a partial-success error indicating CRM customer exists but invitation failed.

Validation Rules

Not Available beyond the business rules above (inherits Lead/Customer validation upstream).

Error Handling

- 409 — lead already converted without a valid customer link (integrity conflict).
- Partial-success error object when CRM conversion succeeds but ERP invitation fails, explicitly informing the caller that the CRM customer record exists even though ERP provisioning did not complete.
- 500 — unexpected transaction errors (transaction rolled back).

Notifications

Triggers ERP invitation email via CRM Integration Module (`sendErpInvitationEmail`).

Security

Runs within an authenticated CRM session; uses row-level locking (`FOR UPDATE`) to prevent race conditions on concurrent conversion attempts.

Configuration

Uses shared CRM-ERP integration configuration (see §7.12).

Environment Variables

Indirectly depends on `ERP_BASE_URL`, `ERP_API_KEY`, and related integration variables (see §7.12).

Dependencies

Express, PostgreSQL (`pg`) transactions, CRM Integration services.

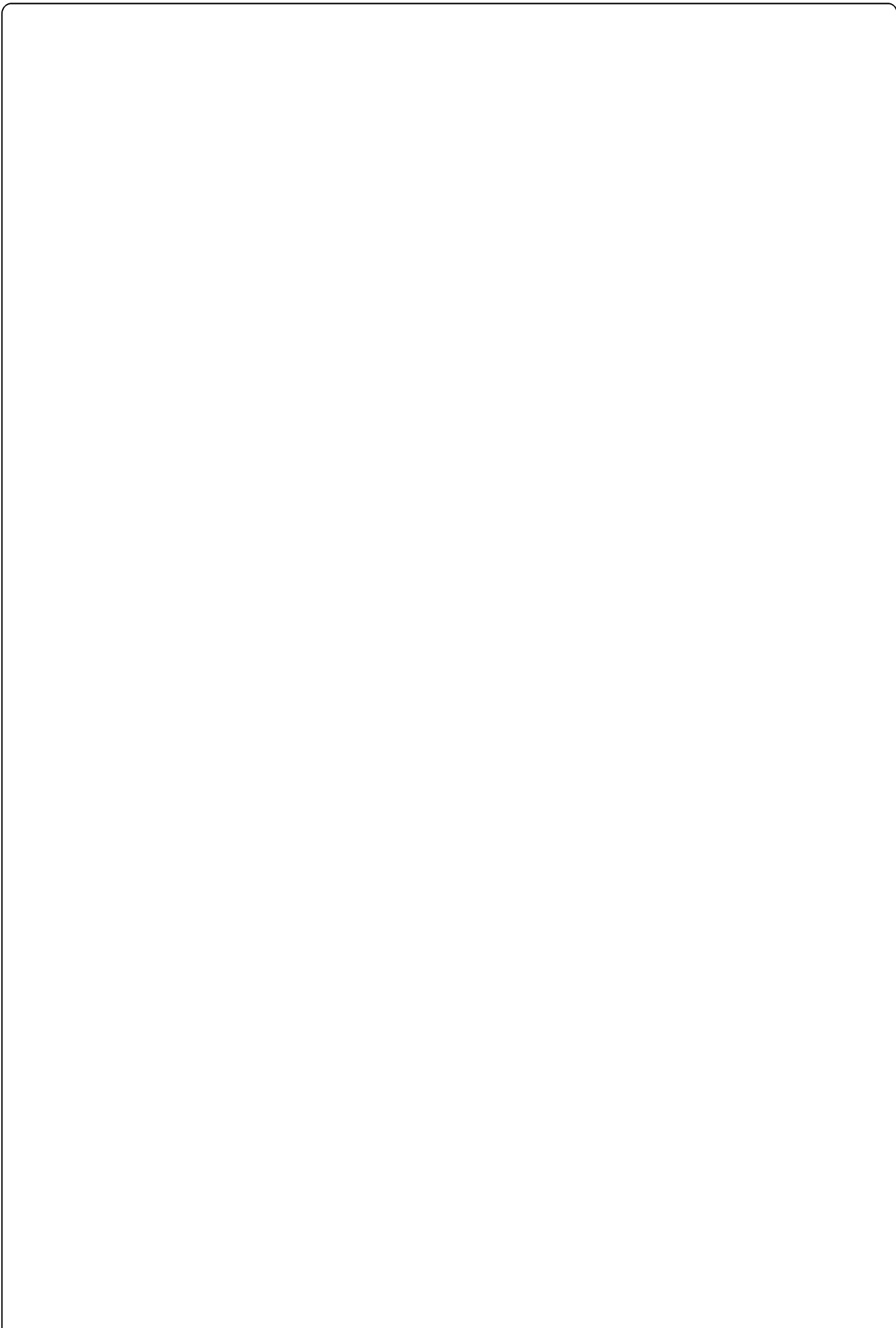
Important Files

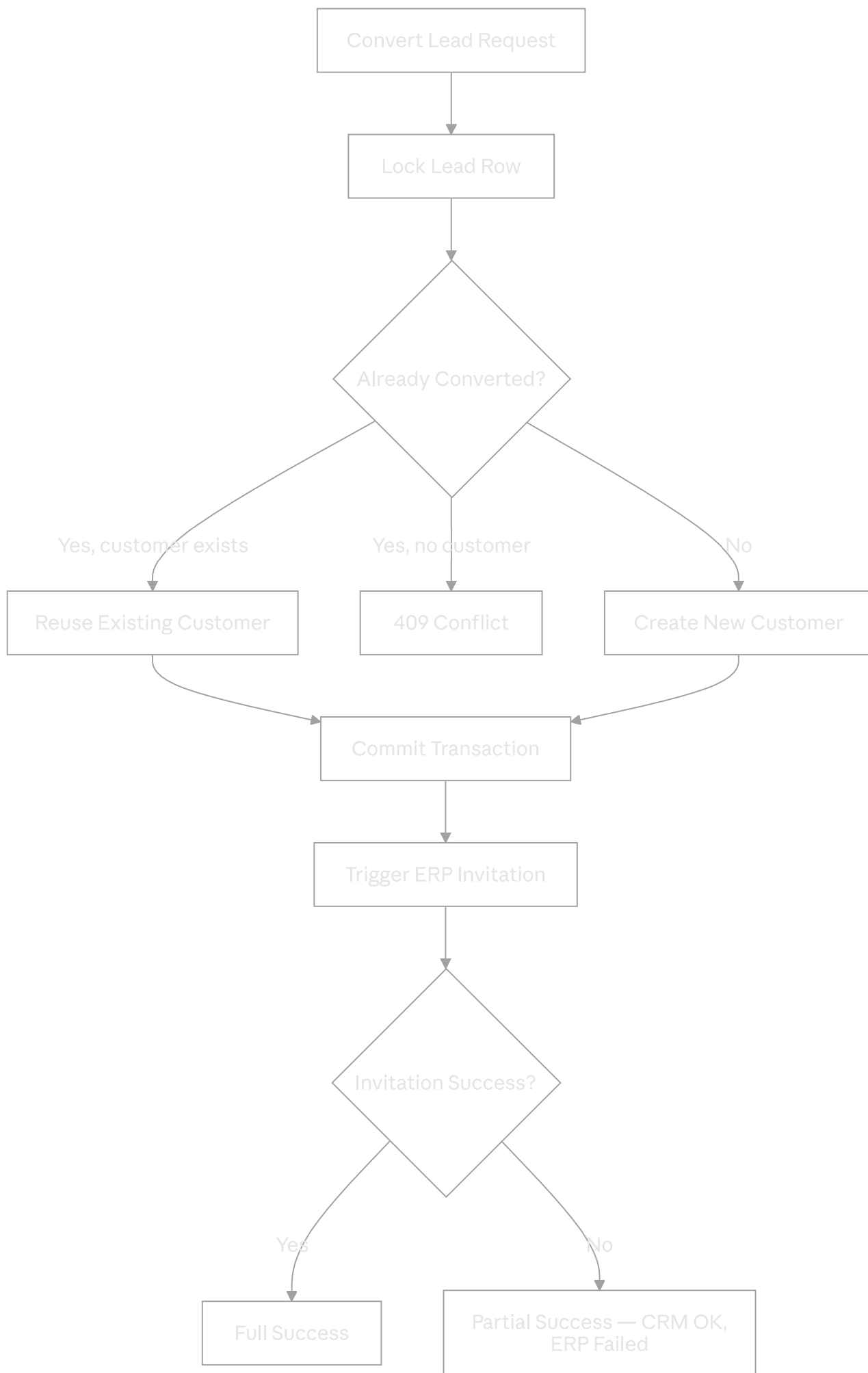
```
controllers/subscriptionController.js),  
integration/services/leadWonIntegrationService.js).
```

Mermaid Sequence Diagram

```
sequenceDiagram  
    participant FE as CRM Frontend  
    participant BE as subscriptionController  
    participant DB as PostgreSQL  
    participant INT as leadWonIntegrationService  
  
    FE->>BE: POST /api/leads/:id/convert-to-customer  
    BE->>DB: BEGIN; SELECT lead FOR UPDATE  
    alt Already converted, no customer  
        BE-->>FE: 409 Conflict  
    else Convertible  
        BE->>DB: INSERT customers (or reuse)  
        BE->>DB: INSERT subscription_history (NEW)  
        BE->>DB: UPDATE leads.is_customer  
        BE->>DB: COMMIT  
        BE->>INT: triggerLeadWonInvitation()  
        INT-->>BE: invitation result (async)  
    end  
    end  
    BE-->>FE: Success / Partial-success response
```

Mermaid Flowchart





Audit Logging

`subscription_history` `NEW` entry serves as the audit record of conversion.

Performance Considerations

Row-level locking (`SELECT ... FOR UPDATE`) minimizes race conditions but should be monitored for lock contention under high concurrent conversion volume. Not further detailed in source.

Troubleshooting

If CRM cannot send ERP invitations, check `ERP_BASE_URL`, `ERP_API_KEY`, ERP `/api/invitations`, and CRM `crm_erp_customer_mappings.error_payload`.

Expected Outcome

Every won lead becomes exactly one CRM customer and triggers exactly one ERP invitation attempt, with safe partial-failure reporting.

Postconditions

`leads.is_customer = true`; customer linked to lead; ERP invitation attempted.

Assumptions

ERP integration endpoint is reachable at conversion time (else partial-success path is used).

Limitations

Not Available beyond documented partial-success behavior.

Best Practices

Monitor `crm_erp_customer_mappings.error_payload` for failed invitations to enable manual remediation.

Future Improvements

Add automated tests for CRM conversion and ERP invitation paths (per source "Future Improvements").

References

Source Implementation Analysis §"CRM Modules → Subscription Conversion Module";
§"Complete Business Workflows → CRM Lead to Trial to Paid Upgrade".

7.6 Plan Module

Module Name

CRM Plan Module

Purpose

Exposes CRM subscription plans for the CRM admin UI and public-website pricing sync.

Scope

Plan read/update, plan feature replacement. Excludes plan purchase/payment processing (see Public Website Pricing Module, Volume 3).

Business Overview

Plans are centrally managed in the CRM and are the single source of truth for pricing displayed on the public website.

User Roles

CRM admin/subscription-management staff.

Functional Requirements

- FR-1: List all plans with nested features.
- FR-2: Update plan core fields.
- FR-3: Replace plan features on update.

Preconditions

Plan must exist for update operations.

Workflow

1. `GET /api/plans` reads all plans and all plan features, nesting matching features under each plan.
2. `PUT /api/plans/:id` updates plan core fields, deletes old feature rows, and inserts submitted feature names.

System Execution Flow

CRM Frontend (`subscriptionplans.jsx`, `PlanEdit.jsx`) → `routes/planRoutes.js` → `controllers/planController.js` → PostgreSQL (`plans`, `plan_features`) → Response. Public website independently calls `GET /api/plans` directly.

Data Flow

Input (update): plan core fields + feature name list. Output: plan objects with nested `features` array.

API Documentation

Base Path	Route File	Purpose
<code>/api/plans</code>	<code>routes/planRoutes.js</code>	Subscription plan read/update

Method	Endpoint	Description
GET	<code>/api/plans</code>	Returns plans with nested <code>features</code>
PUT	<code>/api/plans/:id</code>	Updates plan core fields and replaces plan features

Database Documentation

- Reads `plans`, `plan_features`.
- Updates `plans`.
- Deletes/inserts `plan_features`.

Business Rules

Public website filters inactive/disabled plans client-side based on `is_active`, `active`, `enabled`, `status`, or `plan_status` fields if present.

Validation Rules

Not Available beyond field presence implied by update operation.

Error Handling

Not Available beyond generic CRM error pattern (400/404/500).

Notifications

Not Available.

Security

Standard CRM session-based access for write operations; `GET /api/plans` is consumed cross-origin by the public website (requires CORS configuration — see Troubleshooting).

Configuration

Not Available beyond shared database configuration.

Environment Variables

`DATABASE_URL`.

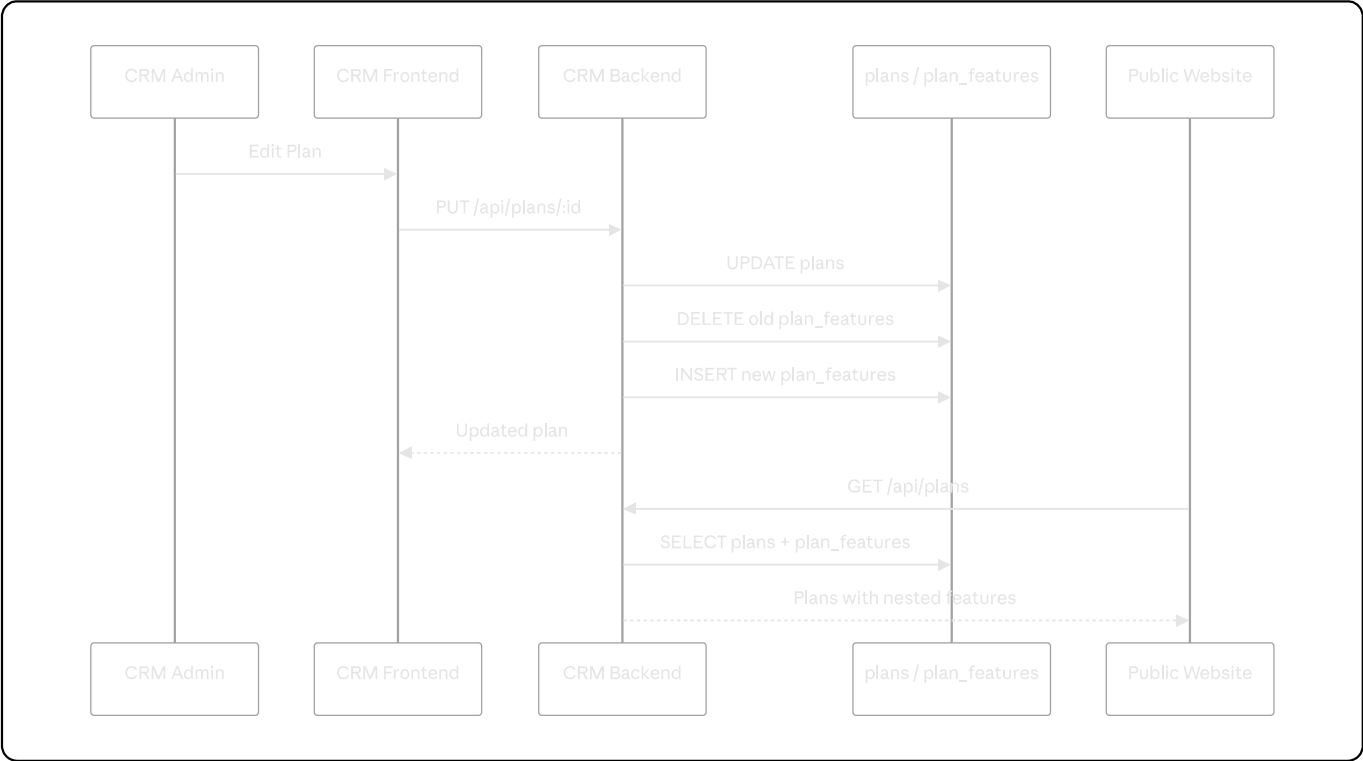
Dependencies

Express, PostgreSQL (`pg`).

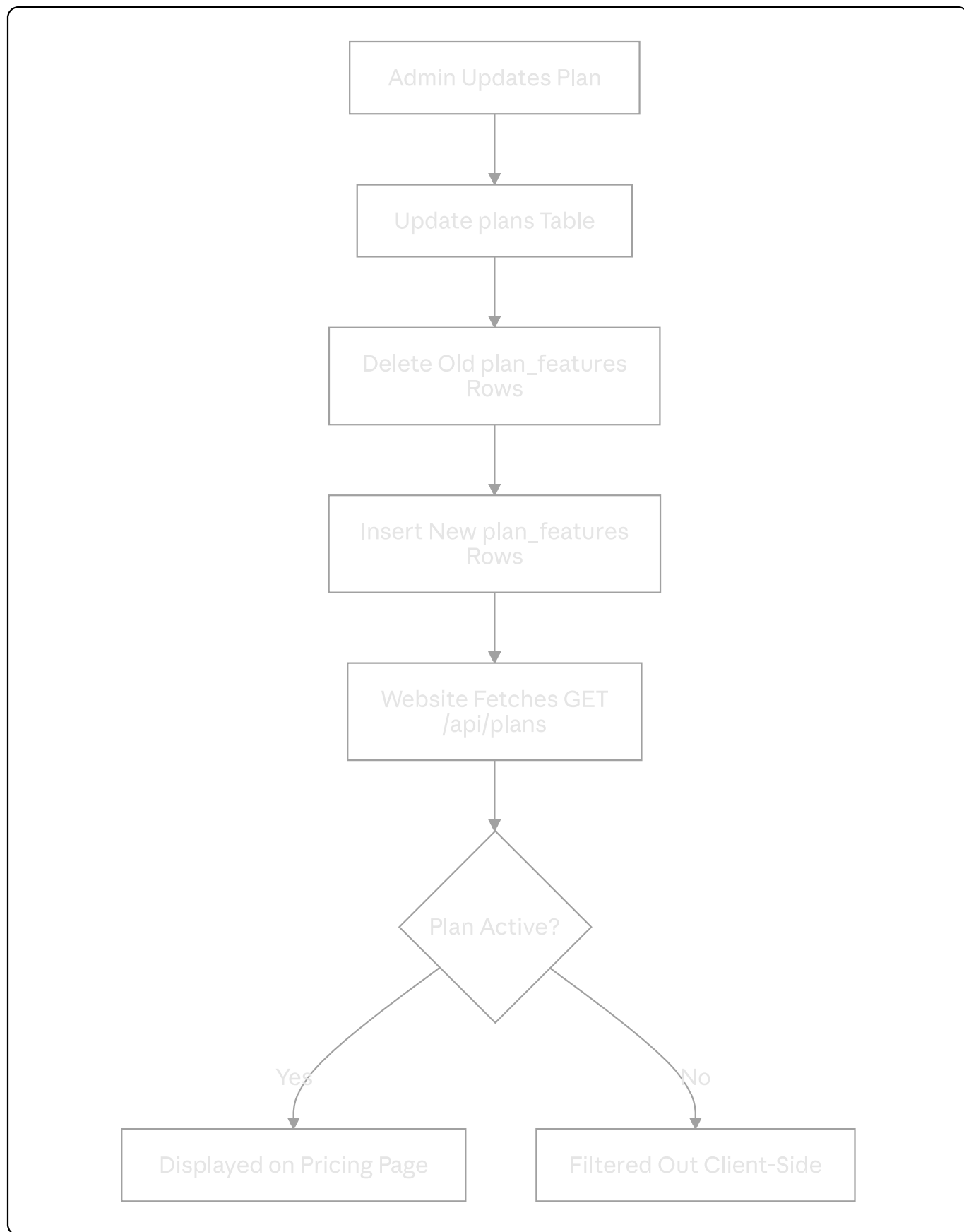
Important Files

`controllers/planController.js`, `routes/planRoutes.js`, CRM Frontend:
`components/subscription/subscriptionplans.jsx`, `components/subscription/PlanEdit.jsx`.

Mermaid Sequence Diagram



Mermaid Flowchart



Audit Logging

Not Available.

Performance Considerations

Not Available.

Troubleshooting

If public plans do not appear, check CRM `GET /api/plans`, CORS, CRM API base URL, and plan active/status fields.

Expected Outcome

Plans edited once in CRM are reliably reflected on the public pricing page, with inactive plans hidden.

Postconditions

`plan_features` rows fully replaced on each update (not merged/patched).

Assumptions

Public website and CRM backend share a consistent understanding of which field name denotes "active" (`is_active`/`active`/`enabled`/`status`/`plan_status`), since the website checks multiple possible field names.

Limitations

No versioning of plan changes described; feature replacement is destructive (delete + insert) rather than diffed.

Best Practices

Confirm at least one active-status field is set correctly before publishing a new plan.

Future Improvements

Standardize API response shapes across modules (per source "Future Improvements").

References

Source Implementation Analysis §"CRM Modules → Plan Module"; §"Complete Business Workflows → Plan Synchronization".

7.7 Task Module

Module Name

CRM Task Module

Purpose

Creates and tracks CRM tasks assigned to employees.

Scope

Task CRUD, status/update tracking, assignee notification.

Business Overview

Tasks are internal work items assigned to CRM employees, often generated automatically as sales follow-ups (e.g., after subscription expiry — see §7.13).

User Roles

CRM employees (assignees); Team/admin staff (assigners).

Functional Requirements

- FR-1: Create task with title, description, assignee, priority, due date.
- FR-2: Track task status.
- FR-3: Record task work updates/notes.
- FR-4: Notify assignee on task creation.

Preconditions

Assignee must resolve to a valid team member via `utils/notifications.getTeamMember`.

Workflow

1. Task created via `controllers/taskController.js`.
2. Assignee resolved via `utils/notifications.getTeamMember`.
3. CRM notification created after task assignment.
4. Task status and update/work-note APIs available thereafter.

System Execution Flow

CRM Frontend → `routes/taskRoutes.js` → `controllers/taskController.js` → PostgreSQL
(`tasks`, `task_updates`, `notifications`) → Response.

Data Flow

Input: title, description, assignee, priority, due date, status. Output: task record; notification record; task update entries.

API Documentation

Base Path	Route File	Purpose
/api/tasks	routes/taskRoutes.js	Task CRUD, status updates, work updates

Database Documentation

- CRUD on `tasks`.
- Writes `task_updates`.
- Writes `notifications`.

Business Rules

- Task title, description, assignee, priority, and due date are required.
- Due date cannot be in the past.
- An employee can view/add task updates only for their assigned task when `employee_id` is supplied in query/body.

Validation Rules

- Priority must be `High`, `Medium`, or `Low`.
- Status must be one of `Open`, `In Progress`, `Completed`, `Done`, `Closed`.
- Title length 3-120 characters.
- Description minimum 10 characters.

Error Handling

Standard CRM error pattern: 400 (validation), 404 (not found), 500 (unexpected). Explicit conflict codes not documented for this module.

Notifications

CRM notification created after task assignment (see §7.9).

Security

Employee-scoped access to task updates via `employee_id` check.

Configuration

Not Available.

`DATABASE_URL`).

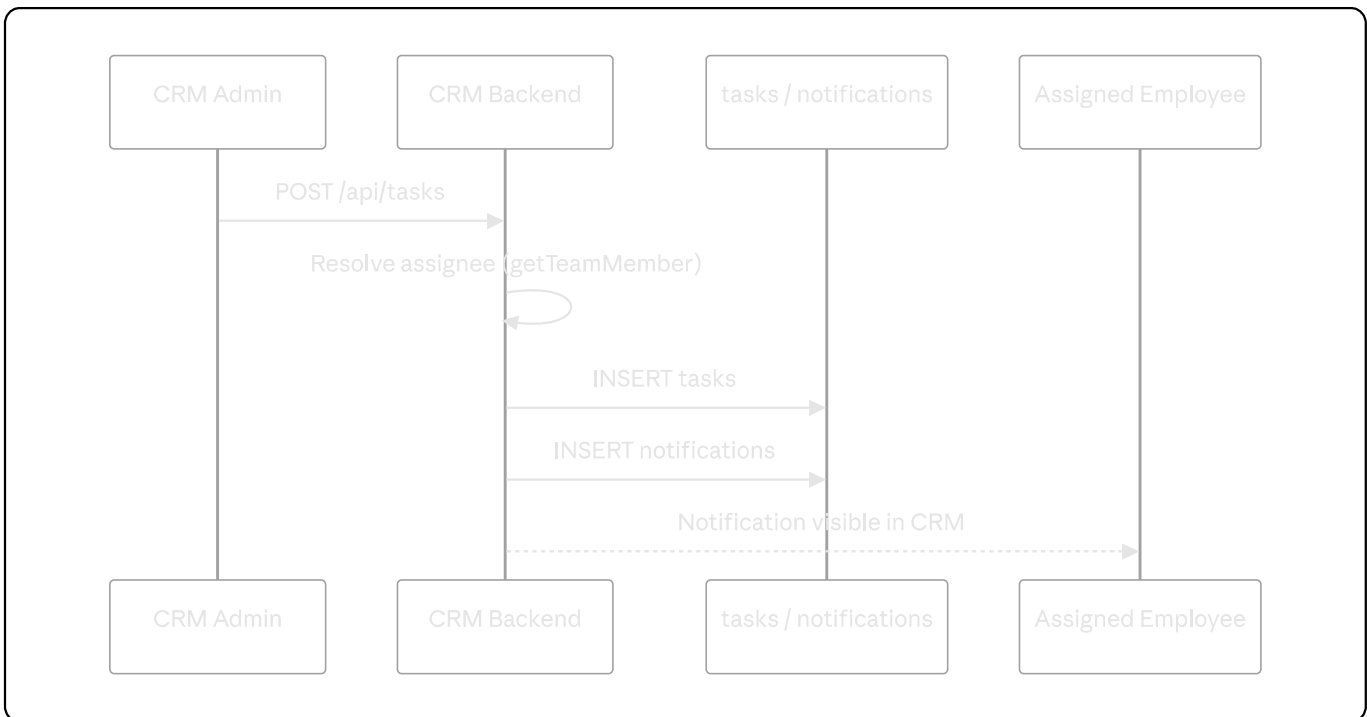
Dependencies

Express, PostgreSQL (`pg`).

Important Files

`controllers/taskController.js`, `routes/taskRoutes.js`, `utils/notifications.js`.

Mermaid Sequence Diagram



Mermaid Flowchart

Audit Logging

`task_updates` provides a work-note trail per task.

Performance Considerations

Not Available.

Troubleshooting

Not Available beyond standard validation errors.

Expected Outcome

Assigned tasks are tracked to completion with a visible notification trail for assignees.

Postconditions

Notification created for assignee; task persisted with initial status.

Assumptions

`employee_id` scoping is consistently enforced across all task-update endpoints (implied, not exhaustively documented).

Limitations

Not Available.

Best Practices

Keep due dates realistic; validate assignee department alignment where relevant to task subject matter (not enforced by system per source).

Future Improvements

Not Available (no module-specific improvement listed).

References

Source Implementation Analysis §"CRM Modules → Task Module".

7.8 Ticket Module

Module Name

CRM Ticket Module

Purpose

Manages CRM support/workspace tickets, assignment, status, history, and stats.

Scope

Ticket CRUD, ticket numbering, department-matched assignment, status history, notifications.

Business Overview

Tickets represent incidents or requests logged for CRM support workflows, each uniquely numbered per type.

User Roles

CRM Support/Technical department staff.

Functional Requirements

- FR-1: Create tickets of type `incident` or `request`.
- FR-2: Auto-generate sequential ticket numbers per type.
- FR-3: Assign ticket to a team member within the correct department.
- FR-4: Track status changes with history.
- FR-5: Notify on assignment and status change.

Preconditions

Assigned member must exist; if department supplied, assigned member must belong to that department.

Workflow

1. `controllers/ticketController.js` generates ticket number: `TIK-INC-0001` or `TIK-REQ-0001` using `ticket_number_counters`.
2. Validates ticket type as `incident` or `request`.
3. Resolves assignee and validates department match.
4. Writes status history when state changes.
5. Creates notifications for assignment and status change.
6. Setting status to `Resolved` sets `resolved_at`.

System Execution Flow

CRM Frontend (ticket UI) → `routes/ticketRoutes.js` → `controllers/ticketController.js`
→ PostgreSQL (`tickets`, `ticket_number_counters`, `ticket_history`, `notifications`) →

Response.

Data Flow

Input: caller, description, ticket type, urgency, department, assignee, due date. Output: ticket record with generated number; ticket history entries; notifications.

API Documentation

Base Path	Route File	Purpose
<code>/api/tickets</code>	<code>routes/ticketRoutes.js</code>	Ticket CRUD, assignment, status, history, stats

Database Documentation

- CRUD on `tickets`.
- Reads/writes `ticket_number_counters`.
- Writes `ticket_history`.
- Writes `notifications`.

Business Rules

- Caller and description are required.
- Assigned member must exist.
- Assigned member must belong to selected department if department is supplied.
- `Resolved` status sets `resolved_at`.

Validation Rules

Inherits caller/description-required rule above; further field-level rules Not Available.

Error Handling

Standard CRM error pattern: 400/404/409(implied for department mismatch)/500.

Explicit status codes beyond those in the Business Rules: Not Available.

Notifications

Notifications created for ticket assignment and status change (see §7.9).

Security

Department-scoped assignment validation.

Configuration

Not Available.

Environment Variables

DATABASE_URL.

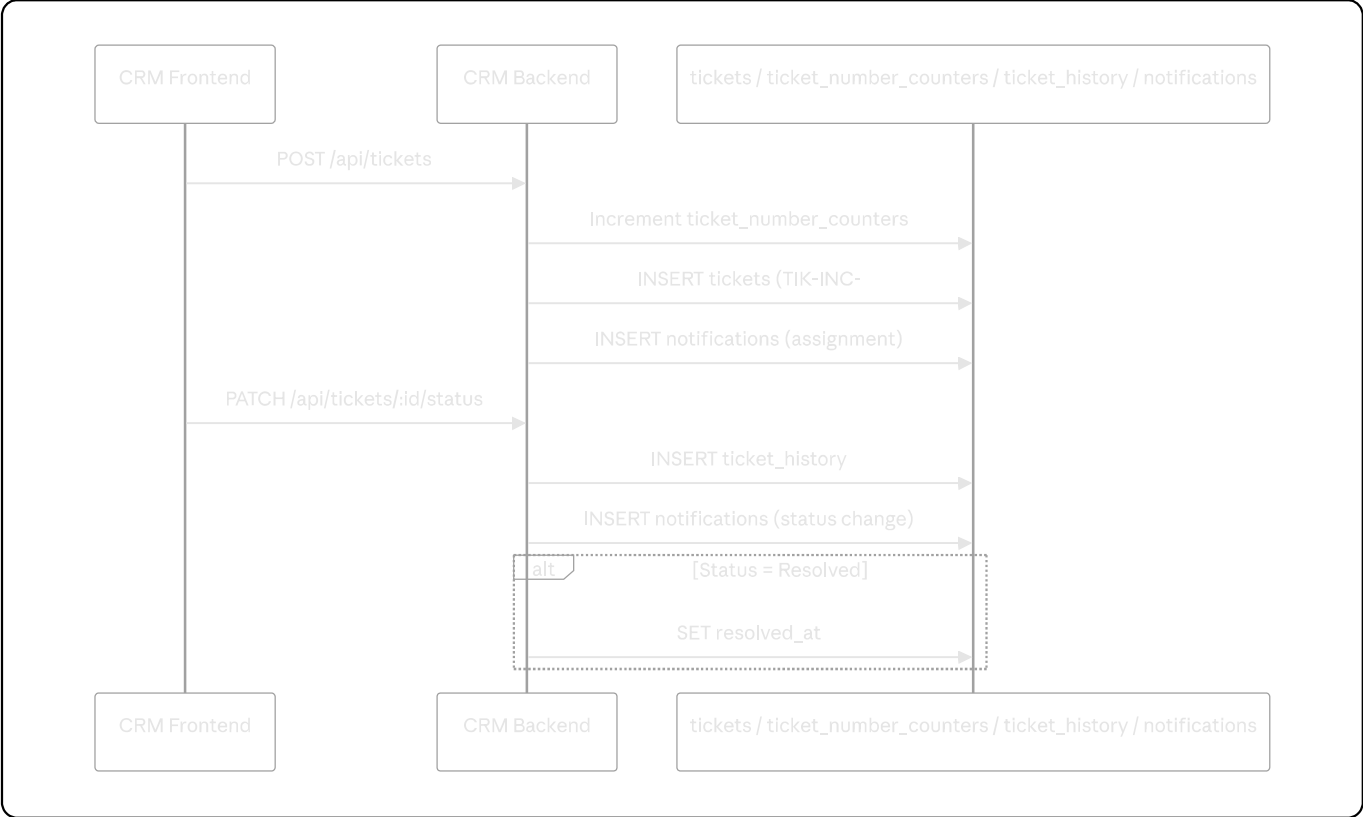
Dependencies

Express, PostgreSQL (pg).

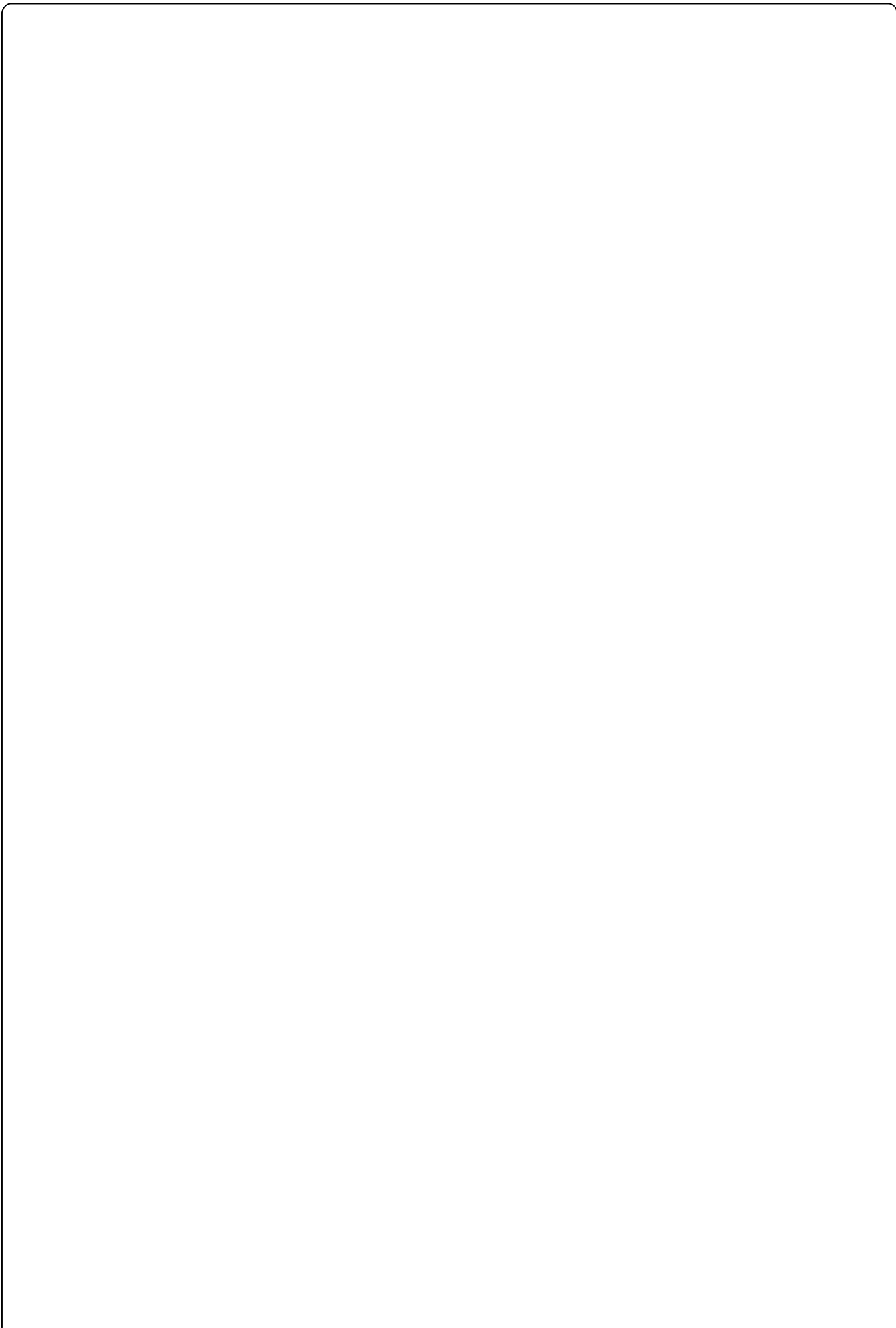
Important Files

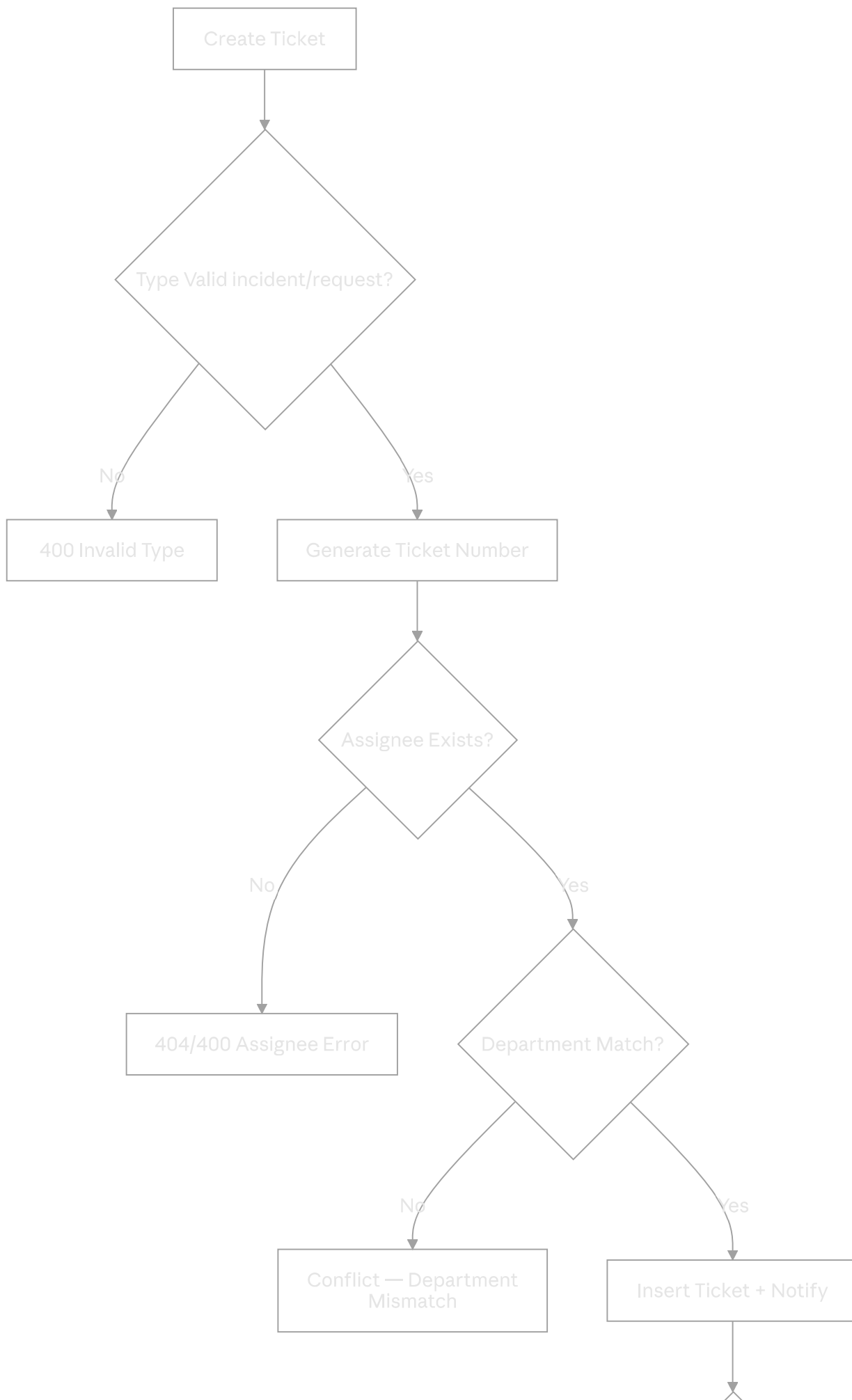
controllers/ticketController.js, routes/ticketRoutes.js, CRM Frontend:
components/tickets/*, components/Employee/Ticket.jsx, Pages/Workspacepage.jsx.

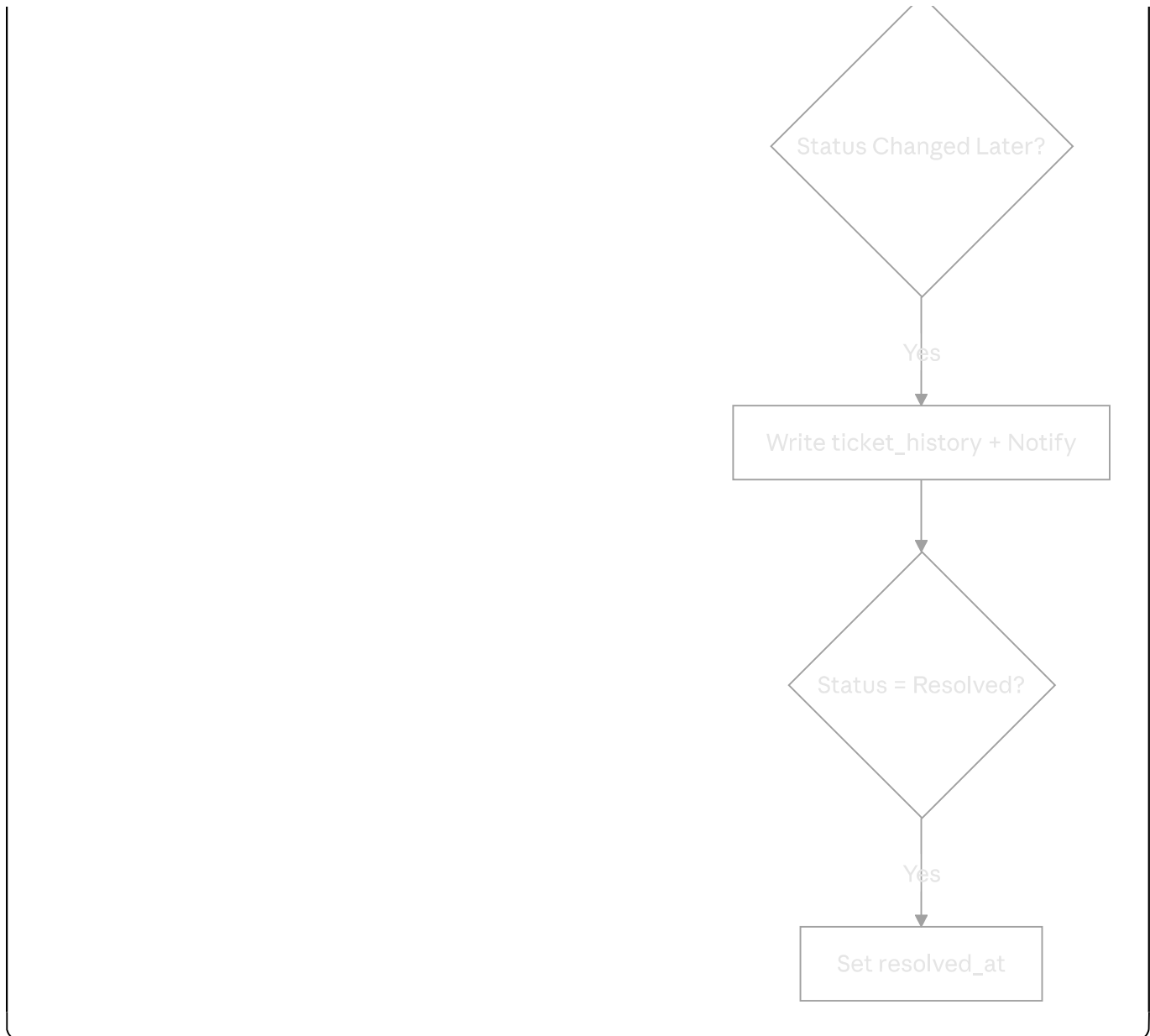
Mermaid Sequence Diagram



Mermaid Flowchart







Audit Logging

`ticket_history` provides the audit trail of status/worknote/follow-up changes.

Performance Considerations

Not Available.

Troubleshooting

Not Available beyond documented validation/assignment rules.

Expected Outcome

Support tickets are uniquely numbered, correctly routed to department-matched staff, and fully auditable through resolution.

Postconditions

Ticket number reserved in `ticket_number_counters`; `resolved_at` set upon resolution.

Assumptions

Department field on team members is authoritative for assignment validation.

Limitations

Not Available.

Best Practices

Always supply department on ticket creation to enable assignment validation and reduce misrouted tickets.

Future Improvements

Not Available (no module-specific improvement listed).

References

Source Implementation Analysis §"CRM Modules → Ticket Module".

7.9 Notification Module

Module Name

CRM Notification Module

Purpose

Stores and exposes CRM employee notifications.

Scope

Notification listing, unread count, mark-as-read.

Business Overview

Central in-app notification store consumed by Task and Ticket modules (and other CRM events) to alert employees.

User Roles

All CRM employees.

Functional Requirements

- FR-1: List notifications for an employee.
- FR-2: Return unread count.
- FR-3: Mark notifications as read.

Preconditions

Notification records must exist for the querying employee.

Workflow

1. `controllers/notificationController.js` and `utils/notifications.js` support listing, unread count, and marking notifications as read.

System Execution Flow

Producing module (Task/Ticket/etc.) → `utils/notifications.js` → INSERT into `notifications` → CRM Frontend polls/fetches via `routes/notificationRoutes.js` → `controllers/notificationController.js` → Response.

Data Flow

Input (write): employee ID, message, source reference. Input (read): employee ID. Output: notification list; unread count; read-status confirmation.

API Documentation

Base Path	Route File	Purpose
<code>/api/notifications</code>	<code>routes/notificationRoutes.js</code>	CRM notifications

Database Documentation

Reads/writes `notifications`.

Business Rules

Not Available beyond producer-driven creation described in Task/Ticket modules.

Validation Rules

Not Available.

Error Handling

Standard CRM error pattern (400/404/500) — not itemized further in source.

Notifications

This module is the notification store itself.

Security

Employee-scoped read access (implied).

Configuration

Not Available.

Environment Variables

`DATABASE_URL`.

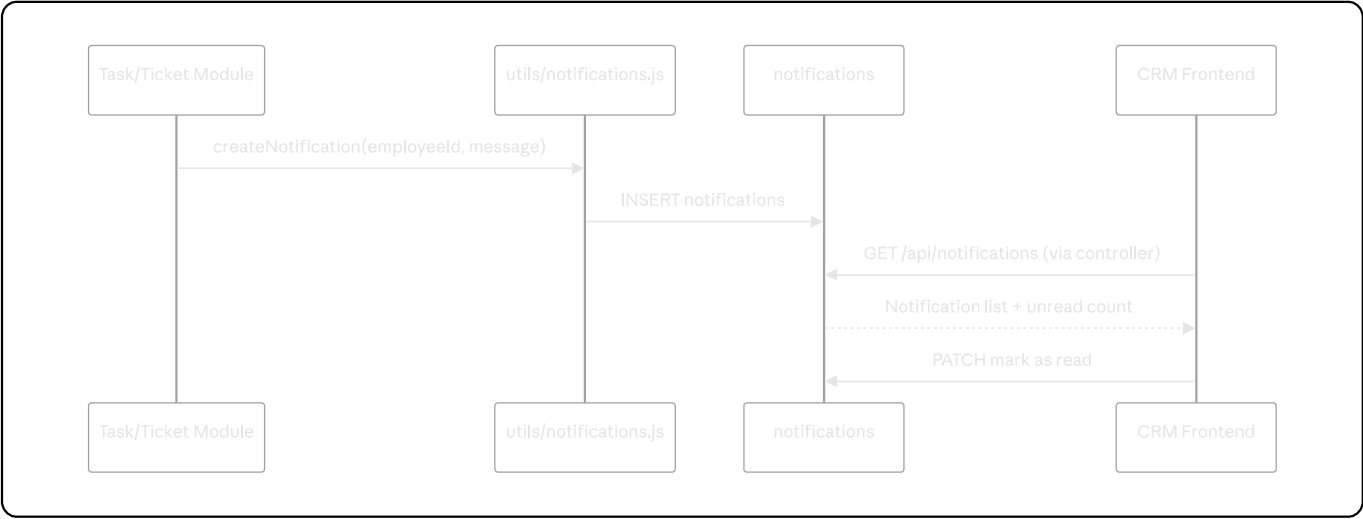
Dependencies

Express, PostgreSQL (`pg`).

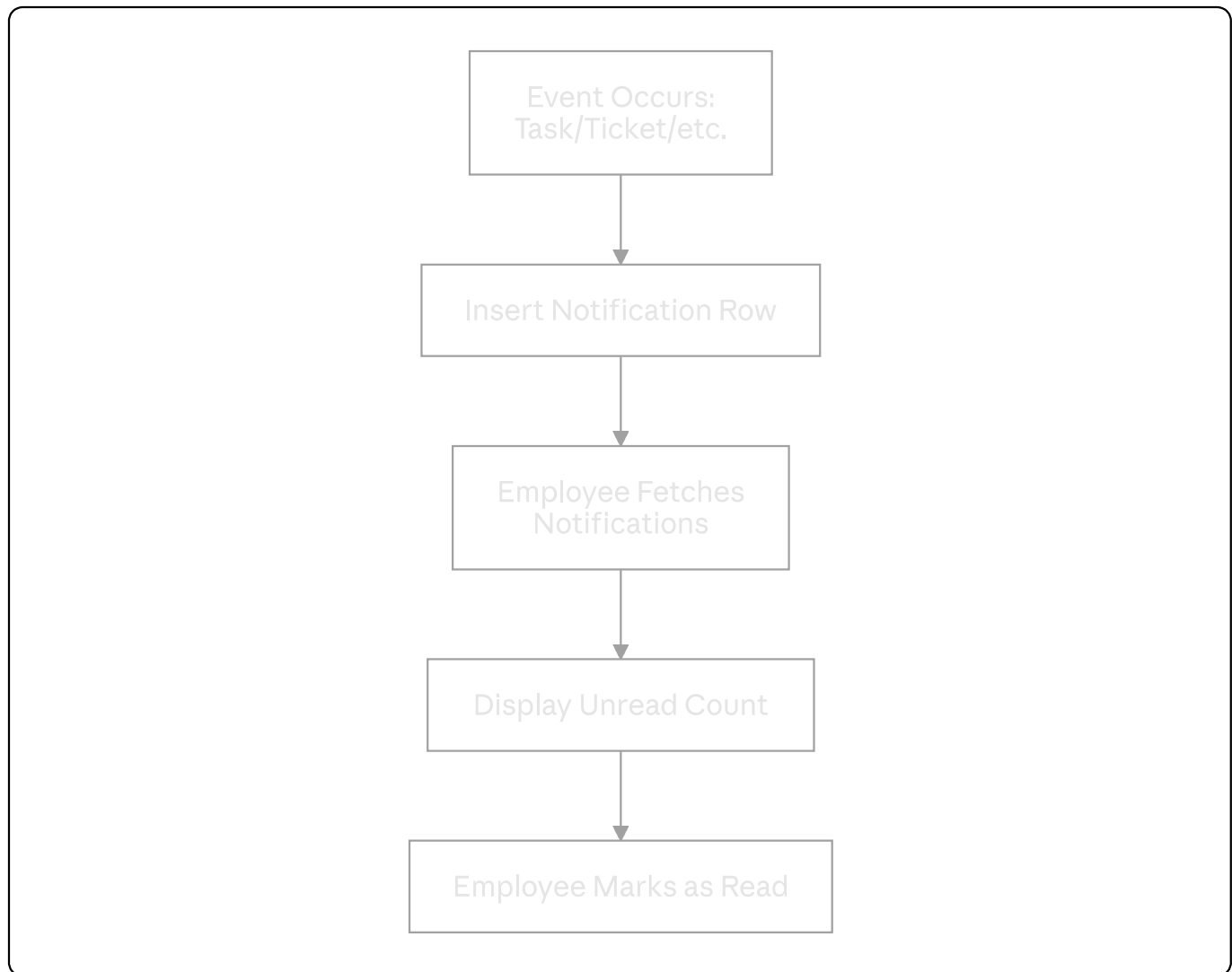
Important Files

`controllers/notificationController.js`, `utils/notifications.js`,
`routes/notificationRoutes.js`.

Mermaid Sequence Diagram



Mermaid Flowchart



Audit Logging

Notification history itself provides a partial activity trail.

Performance Considerations

Not Available.

Troubleshooting

Not Available.

Expected Outcome

Employees receive timely in-app notifications for relevant CRM events.

Postconditions

Read state updated on employee action.

Assumptions

Notifications are employee-scoped (not broadcast) based on producer usage pattern.

Limitations

Not Available (no push/real-time delivery mechanism described for CRM, unlike ERP frontend's Socket.IO client — see Volume 3).

Best Practices

Periodically archive/prune old read notifications (not described as implemented; recommended operational practice).

Future Improvements

Not Available.

References

Source Implementation Analysis §"CRM Modules → Notification Module".

7.10 Department Module

Module Name

CRM Department Module

Purpose

Powers department dashboards, department member management, performance metrics, department health, and sales dashboard data.

Scope

Department summary, stats, performance (including year-over-year), health, sales dashboard, sales member due-today tasks, department member CRUD.

Business Overview

Provides managers visibility into departmental performance and health across Sales, Support, Marketing, and Technical workspaces.

User Roles

Department managers/admins; department members (workspace views).

Functional Requirements

- FR-1: Provide department summary and stats.

- FR-2: Provide performance metrics and historical performance by year.
- FR-3: Provide department health overview.
- FR-4: Provide sales dashboard and due-today task lists.
- FR-5: CRUD department members.

Preconditions

Not Available beyond valid department identifier.

Workflow

1. `controllers/departmentController.js` aggregates CRM employee/task/ticket/customer/lead data depending on the requested endpoint.
2. `routes/departmentRoutes.js` exposes summary, stats, performance, performance-years, health, sales dashboard, sales member due-today tasks, and department member CRUD.

System Execution Flow

CRM Frontend (department pages) → `routes/departmentRoutes.js` → `controllers/departmentController.js` → PostgreSQL (multiple tables: employees/tasks/tickets/customers/leads) → aggregated Response.

Data Flow

Input: department identifier, optional year filter. Output: aggregated stats/health/performance objects; department member records.

API Documentation

Base Path	Route File	Purpose
<code>/api/departments</code>	<code>routes/departmentRoutes.js</code>	Department dashboards, members, health, performance

Database Documentation

Uses CRM employee/task/ticket/customer/lead data depending on endpoint (cross-module read aggregation).

Business Rules

Not Available beyond aggregation scope described above.

Validation Rules

Not Available.

Error Handling

Standard CRM error pattern (400/404/500) — not itemized further in source.

Notifications

Not Available.

Security

Presumed department-manager-level access for member CRUD; not explicitly detailed in source.

Configuration

Not Available.

Environment Variables

DATABASE_URL.

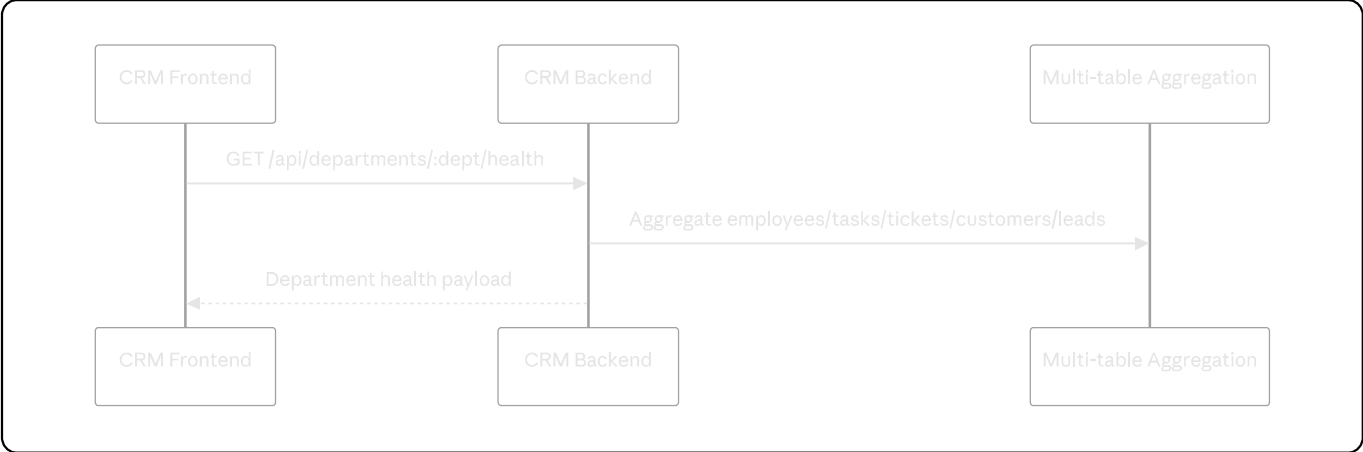
Dependencies

Express, PostgreSQL (pg).

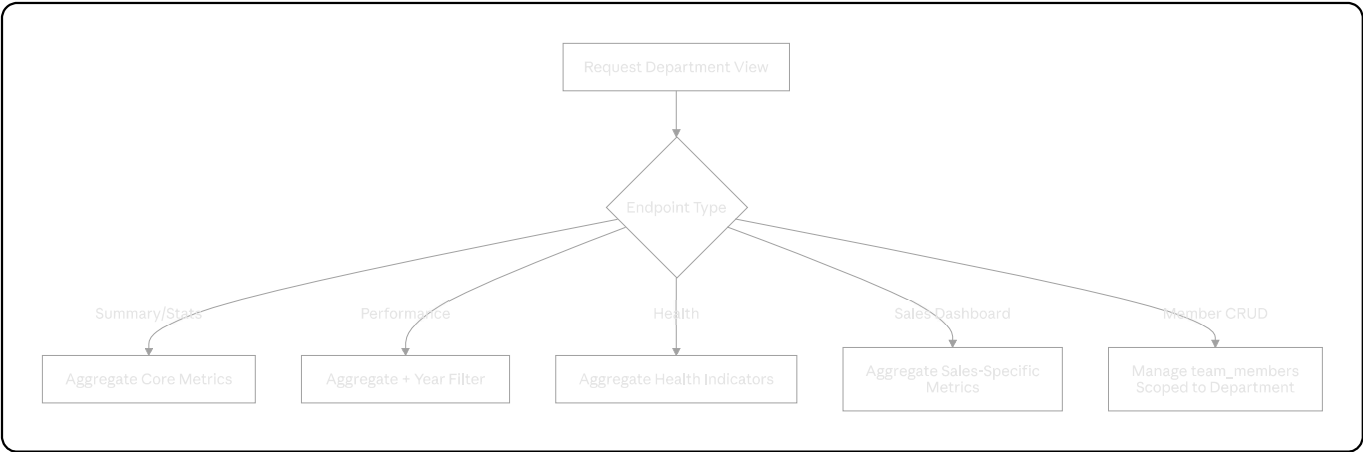
Important Files

controllers/departmentController.js, routes/departmentRoutes.js, CRM Frontend: Pages/department/*, components/department/*.

Mermaid Sequence Diagram



Mermaid Flowchart



Audit Logging

Not Available.

Performance Considerations

Aggregation across multiple tables (employees/tasks/tickets/customers/leads) may require indexing/optimization at scale; not detailed in source.

Troubleshooting

Not Available.

Expected Outcome

Managers obtain consolidated visibility into department performance, health, and staffing.

Postconditions

Not Available.

Assumptions

Department names used consistently across `team_members`, `tasks`, and `tickets` tables to enable accurate aggregation.

Limitations

Not Available.

Best Practices

Keep department naming consistent across all source tables to avoid aggregation gaps.

Future Improvements

Not Available.

References

Source Implementation Analysis §"CRM Modules → Department Module".

7.11 Dashboard Module

Module Name

CRM Dashboard Module

Purpose

Provides CRM dashboard summary statistics.

Scope

Single summary statistics endpoint.

Business Overview

Gives CRM users a top-level overview of key metrics on login.

User Roles

All CRM employees (dashboard landing page).

Functional Requirements

- FR-1: Provide aggregated CRM statistics via `GET /api/dashboard/stats`.

Preconditions

None beyond authenticated session.

Workflow

1. `controllers/dashboardController.js` aggregates statistics.
2. `routes/dashboardRoutes.js` exposes `GET /api/dashboard/stats`.

System Execution Flow

CRM Frontend dashboard → `routes/dashboardRoutes.js` → `controllers/dashboardController.js` → PostgreSQL (aggregation) → Response.

Data Flow

Output: summary statistics object (exact fields Not Available in source).

API Documentation

Base Path		Route File	Purpose
/api/dashboard		routes/dashboardRoutes.js	CRM dashboard stats
Method	Endpoint	Description	
GET	/api/dashboard/stats	CRM dashboard summary statistics	

Database Documentation

Not Available (specific tables aggregated not enumerated beyond general CRM dataset).

Business Rules

Not Available.

Validation Rules

Not Available.

Error Handling

Standard CRM error pattern (500 for unexpected errors) — not itemized further.

Notifications

Not Available.

Security

Authenticated CRM session required (implied).

Configuration

Not Available.

Environment Variables

DATABASE_URL.

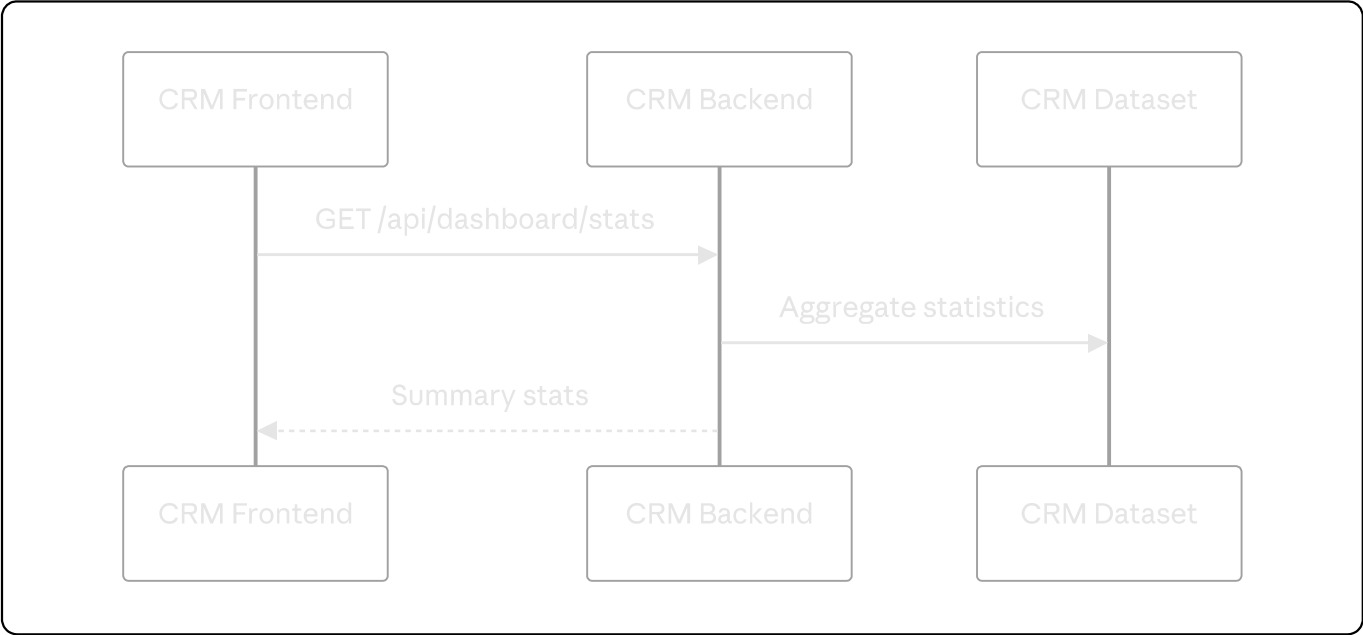
Dependencies

Express, PostgreSQL (pg).

Important Files

controllers/dashboardController.js, routes/dashboardRoutes.js.

Mermaid Sequence Diagram



Mermaid Flowchart

```
flowchart TD
    A[Load Dashboard] --> B[GET /api/dashboard/stats]
    B --> C[Aggregate CRM Metrics]
    C --> D[Render Summary Widgets]
```

Audit Logging

Not Available.

Performance Considerations

Not Available.

Troubleshooting

Not Available.

Expected Outcome

CRM users see an accurate real-time summary on login.

Postconditions

Not Available.

Assumptions

Dashboard stats reflect near-real-time data (no caching layer described).

Limitations

Not Available.

Best Practices

Not Available.

Future Improvements

Not Available.

References

Source Implementation Analysis §"CRM Modules → Dashboard Module".

7.12 CRM Integration Module

Module Name

CRM Integration Module

Purpose

Bridges CRM customer/lead/subscription lifecycle with ERP invitations and account status.

Scope

Lead-won invitation triggering, direct pricing purchase customer handling, ERP status event ingestion. Excludes ERP-side handling (Volume 2).

Business Overview

This is the sole authorized channel through which CRM and ERP exchange state, protected by API-key authentication and idempotency guarantees.

User Roles

System-to-system (CRM backend ↔ ERP backend); indirectly triggered by CRM Sales staff (via lead conversion) and website customers (via direct purchase).

Functional Requirements

- FR-1: Build and send ERP invitation payload on lead conversion.

- FR-2: Prevent duplicate invitations via idempotency mapping.
- FR-3: Send invitation email to the customer.
- FR-4: Receive and apply ERP status events to CRM customer records.
- FR-5: Handle direct website pricing purchase customer creation/reuse.

Preconditions

`ERP_BASE_URL` and `ERP_API_KEY` must be configured; `crm_erp_customer_mappings` table available.

Workflow

1. Lead conversion builds an ERP invitation payload.
2. `crm_erp_customer_mappings` is checked for an existing completed or pending mapping.
3. If no completed mapping exists, CRM sends an invitation request to ERP.
4. ERP returns invitation/customer IDs.
5. CRM sends invitation email using `sendErpInvitationEmail`.
6. CRM updates the mapping and customer ERP reference fields.
7. ERP later sends status events to CRM through integration routes.

System Execution Flow

`subscriptionController` (post-commit) → `leadWonIntegrationService` → `erpApiClient` (HTTP, API-key auth, retry) → ERP `/api/invitations` → response → `crm_erp_customer_mappings` upsert → `sendErpInvitationEmail`. Reverse direction: ERP → `POST /api/integration/customer-status-events` → `customerStatusSyncService` → `customers` update.

Data Flow

CRM → ERP invitation payload: `source: CRM`, `event: LEAD_WON`, `idempotencyKey`, `crmLeadId`, `crmCustomerId`, `customer.name`, `customer.companyName`, `customer.email`, `customer.phone`, `customer.address`, `customer.location`, `customer.channel`, `customer.subscriptionPlan`, `customer.paymentStatus`. ERP stores the payload in `CrmInvitation.requestPayload`.

ERP → CRM subscription status data: `eventId`, `crmCustomerId`, `crmLeadId`, `erpCustomerId`, `status`, trial dates or purchase date, plan when applicable. CRM stores status events in `crm_erp_status_events` and updates `customers`.

Website pricing purchase data (relayed via ERP to CRM): `purchaseFlow`, `userId`, `customerId`, `crmCustomerId`, `erpCustomerId`, `name`, `companyName`, `email`, `phone`, `plan`,

paymentId).

API Documentation

Base Path	Route File	Purpose
/api/integration	integration/routes/integrationRoutes.js	CRM-ERP integration endpoints

Method	Endpoint	Description
POST	/api/integration/customers/direct-pricing-purchase	Creates/reuses a CRM for direct website purchase handleDirectPricingPurchase
POST	/api/integration/customer-status-events	Receives ERP status change events syncs CRM state
GET	/api/integration/registration/invitations/:invitationId	Exposes CRM invitation registration

Database Documentation

- Reads/writes crm_erp_customer_mappings (idempotency, mapping state, error_payload).
- Reads/writes crm_erp_status_events (idempotent ERP-origin events).
- Updates customers (ERP reference fields, status).

Business Rules

- Idempotency key crm-lead-{lead.id}-customer-{customer.id} prevents duplicate invitations.
- Completed invitation mappings are reused.
- Pending mappings are not duplicated.
- Direct pricing purchase uses ensurePaidPricingCustomer through directPricingPurchaseService.

Validation Rules

Not Available beyond idempotency-key uniqueness enforcement.

Error Handling

Integration controller wraps ERP errors and includes `(code)`/`(details)`. Conversion returns a specific partial-success message when CRM conversion succeeded but ERP invitation failed (see §7.5).

Notifications

`sendErpInvitationEmail` sends the ERP invitation email to the customer.

Security

- API-key middleware (`(integration/middleware/apiKeyAuth.js)`) validates configured API key for protected sync endpoints.
- Outbound calls use `(integration/services/erpApiClient.js)` with configurable timeout and retry.

Configuration

`(integration/config/integrationConfig.js)` centralizes ERP base URL, API key, path overrides, timeout, and retry settings.

Environment Variables

`(ERP_BASE_URL)`, `(ERP_API_KEY)`, `(ERP_WEBHOOK_SECRET)`, `(ERP_API_TIMEOUT_MS)`,
`(ERP_API_RETRY_ATTEMPTS)`, `(ERP_API_RETRY_DELAY_MS)`, `(ERP_INVITATION_PATH)`,
`(ERP_CUSTOMER_PATH)`, `(ERP_CUSTOMER_STATUS_PATH)`, `(CRM_INVITATION_FRONTEND_URL)`,
`(CRM_FRONTEND_URL)`.

Dependencies

Express, Axios (outbound HTTP), PostgreSQL (`(pg)`), Nodemailer.

Important Files

`(integration/controllers/integrationController.js)`,
`(integration/routes/integrationRoutes.js)`, `(integration/services/erpApiClient.js)`,
`(integration/services/leadWonIntegrationService.js)`,
`(integration/services/customerStatusSyncService.js)`,
`(integration/services/directPricingPurchaseService.js)`,
`(integration/middleware/apiKeyAuth.js)`, `(integration/config/integrationConfig.js)`.

Mermaid Sequence Diagram

```
sequenceDiagram
    participant CRMB as CRM Backend
```

```

participant Map as crm_erp_customer_mappings
participant ERP as ERP Backend
participant Mail as Email Service

CRMB->>Map: Check existing mapping (idempotency key)
alt Completed mapping exists
    Map-->>CRMB: Reuse existing invitation
else No completed mapping
    CRMB->>ERP: POST /api/invitations (API key)
    ERP-->>CRMB: invitationId, erpCustomerId
    CRMB->>Map: Upsert mapping
    CRMB->>Mail: sendErpInvitationEmail
end
ERP->>CRMB: POST /api/integration/customer-status-events
CRMB->>CRMB: customerStatusSyncService updates customers

```

Mermaid Flowchart

```

flowchart TD
    A[Lead Won / Direct Purchase] --> B{Mapping Exists?}
    B -->|Completed| C[Reuse Mapping]
    B -->|Pending| D[Await – Do Not Duplicate]
    B -->|None| E[Send Invitation to ERP]
    E --> F{ERP Success?}
    F -->|Yes| G[Upsert Mapping + Send Email]
    F -->|No| H[Store error_payload – Partial Success]
    I[ERP Status Event Received] --> J[Validate + Store in crm_erp_status_events]
    J --> K[Update customers Table]

```

Audit Logging

`crm_erp_customer_mappings.error_payload` and `crm_erp_status_events` provide an audit/error trail for integration activity.

Performance Considerations

Outbound ERP calls use configurable retry (`ERP_API_RETRY_ATTEMPTS`, `ERP_API_RETRY_DELAY_MS`) and timeout (`ERP_API_TIMEOUT_MS`) to tolerate transient ERP unavailability.

Troubleshooting

If CRM cannot send ERP invitations, check `ERP_BASE_URL`, `ERP_API_KEY`, `ERP /api/invitations`, and CRM `crm_erp_customer_mappings.error_payload`.

Expected Outcome

Reliable, idempotent, auditable synchronization of customer lifecycle state between CRM

and ERP.

Postconditions

Mapping record reflects final invitation state (`completed`)/(`pending`)/error); customer ERP reference fields populated once available.

Assumptions

ERP and CRM API keys are correctly and securely provisioned in both environments.

Limitations

Legacy webhook path (ERP side) uses internal fetch to localhost superadmin route — documented as a limitation/gap (see Volume 2, ERP Webhook Module).

Best Practices

Rotate (`ERP_API_KEY`)/(`CRM_API_KEY`) periodically; monitor mapping error payloads for stuck/pending invitations.

Future Improvements

Replace legacy webhook internal localhost fetch with direct service invocation (per source "Future Improvements", applies jointly with ERP module).

References

Source Implementation Analysis §"CRM Modules → CRM Integration Module"; §"Data Flow"; §"Security Implementation".

7.13 CRM Scheduler Modules (Follow-Up Reminder & Subscription Expiry)

Module Name

CRM Scheduler Modules — Follow-Up Reminder and Paid Subscription Expiry

Purpose

Automates lead follow-up reminders and the paid subscription renewal-reminder/expiry lifecycle.

Scope

Two related scheduled jobs: (a) lead follow-up reminders, (b) paid subscription renewal reminders and expiry processing including ERP deactivation and CRM follow-up task creation.

Business Overview

Ensures sales follow-ups on leads are not missed and that paying customers are proactively reminded before renewal, with automatic account suspension (not deletion) on non-renewal.

User Roles

System-scheduled (no direct user role); output consumed by CRM Sales staff (follow-up tasks) and customers (email reminders).

Functional Requirements

- FR-1: Send lead follow-up reminders on schedule via `node-cron`.
- FR-2: Prevent duplicate follow-up reminders.
- FR-3: Send paid-subscription renewal reminders at 30/7/1 days before renewal (configurable).
- FR-4: Process subscription expiry: email, CRM status update, ERP deactivation, history write, follow-up task creation.
- FR-5: Never delete ERP account or customer data on expiry.

Preconditions

`node-cron` scheduler running within CRM `server.js` process; ERP integration reachable for status update calls.

Workflow

Follow-Up Reminders (`scripts/followUpReminder.js`, `services/followUpReminderService.js`):

1. Scheduled job reads lead follow-up fields.
2. Duplicate prevention via reminder-sent fields and a unique notification index.

Paid Subscription Expiry (`scripts/subscriptionExpiry.js`, `services/subscriptionExpiryService.js`):

1. Default reminder days from `SUBSCRIPTION_REMINDER_DAYS` or `30,7,1`.
2. Renewal reminder sent for paid active customers at configured intervals.
3. `crm_erp_status_events` event IDs reserve processing to prevent duplicate reminders.
4. On expiry: sends expiry email; marks CRM subscription expired; calls ERP status update to disable login; writes `subscription_history` (`EXPIRED`); creates a sales follow-up task.
5. ERP account and customer data are preserved (not deleted).

System Execution Flow

`server.js` starts `scripts/followUpReminder.js` and `scripts/subscriptionExpiry.js` at startup (via `node-cron`, default expression `15 0 * * *` for expiry) → services query `leads`/`customers` → email dispatch → ERP status call → CRM DB writes.

Data Flow

Output (reminders): email to customer/lead; `notifications` entries. Output (expiry): email to customer; `customers` status update; `subscription_history` `EXPIRED` entry; ERP status API call (`isActive=false`); new `tasks` entry (sales follow-up).

API Documentation

Outbound only: CRM → ERP status update call (path per `ERP_CUSTOMER_STATUS_PATH`), via `erpApiClient`). No inbound HTTP endpoint specific to the scheduler itself (invoked by cron, not HTTP).

Database Documentation

- Reads `leads` (follow-up fields), `customers` (renewal date, status).
- Writes `crm_erp_status_events` (event reservation/idempotency).
- Writes `subscription_history` (`EXPIRED`).
- Writes `tasks` (follow-up task).
- Writes `notifications` (reminder-related, where applicable).

Business Rules

- Follow-up reminders are deduplicated via reminder-sent fields and a unique notification index.
- Renewal reminder default schedule: 30, 7, and 1 day(s) before renewal (`SUBSCRIPTION_REMINDER_DAYS`).
- `crm_erp_status_events` event IDs reserve processing to prevent duplicate reminders/expiry actions.
- Expiry disables ERP login but does not delete ERP account or CRM customer data.
- A CRM sales follow-up task is created automatically on expiry.

Validation Rules

Not Available beyond date-based eligibility checks implied by the business rules.

Error Handling

Trial/CRM sync failures can leave events pending for retry (shared characteristic with ERP-side sync per source "Error Handling" section). Specific HTTP-style codes are Not Available since this runs as a background job, not an HTTP endpoint.

Notifications

- Renewal reminder emails (30/7/1-day cadence).
- Expiry email.
- CRM sales follow-up task (functions as an internal notification/action item).

Security

Uses the same `ERP_API_KEY`-authenticated integration channel as the CRM Integration Module for the ERP status update call.

Configuration

- `SUBSCRIPTION_EXPIRY_CRON` — cron expression (default `15 0 * * *`).
- `SUBSCRIPTION_REMINDER_DAYS` — comma-separated reminder day offsets (default `30,7,1`).

Environment Variables

`SUBSCRIPTION_REMINDER_DAYS`, `SUBSCRIPTION_EXPIRY_CRON`, plus shared ERP integration variables (`ERP_BASE_URL`, `ERP_API_KEY`, `ERP_CUSTOMER_STATUS_PATH`, etc.).

Dependencies

`node-cron`, Nodemailer, Axios (via `erpApiClient`), PostgreSQL (`pg`).

Important Files

`scripts/followUpReminder.js`, `scripts/subscriptionExpiry.js`,
`services/followUpReminderService.js`, `services/subscriptionExpiryService.js`,
`integration/services/customerStatusSyncService.js` (follow-up task creation).

Mermaid Sequence Diagram

```
sequenceDiagram
    participant Cron as CRM Scheduler (node-cron)
    participant CRM as CRM DB/Services
    participant Mail as Email Service
    participant ERP as ERP Backend
    participant Sales as Sales Tasks
```

```
Cron->>CRM: Find paid active subscriptions near expiry
CRM->>CRM: Reserve reminder event (crm_erp_status_events)
CRM->>Mail: Send 30/7/1-day reminder
Cron->>CRM: Find expired paid subscriptions
CRM->>CRM: Reserve expiry event
CRM->>Mail: Send expired email
CRM->>CRM: Mark subscription expired (subscription_history: EXPIRED)
CRM->>ERP: PUT customer status isActive=false
CRM->>Sales: Create follow-up task
```

Mermaid Flowchart

```
graph TD
    A[Cron Trigger 15 0 * * *] --> B[Scan Paid Active Customers]
    B --> C{Renewal Date within 30/7/1 days?}
    C -->|Yes, not yet reminded| D[Reserve Event + Send Reminder]
    C -->|No| E[Skip]
    A --> F[Scan for Passed Renewal Dates]
    F --> G{Already Processed?}
    G -->|Yes| E
    G -->|No| H[Reserve Expiry Event]
    H --> I[Send Expiry Email]
    I --> J[Mark Customer Expired + Write History]
    J --> K[Call ERP: Disable Login]
    K --> L[Create Sales Follow-Up Task]
```

Audit Logging

`crm_erp_status_events` and `subscription_history` jointly provide the audit trail for reminder/expiry processing.

Performance Considerations

Idempotent event reservation prevents duplicate processing under concurrent/overlapping cron executions. No further performance detail provided.

Troubleshooting

- If duplicate reminder emails appear, inspect `crm_erp_status_events` event IDs for the affected customer/date.
- If ERP login is disabled unexpectedly, inspect ERP `User.isActive`, `accountStatus`, and CRM expiry events.

Expected Outcome

Customers are proactively reminded before renewal; expired paid subscriptions are

automatically suspended (ERP login disabled) with no data loss, and sales is automatically alerted via a follow-up task.

Postconditions

On expiry: `customers` status = expired/`Subscription Expired`; `subscription_history` has `EXPIRED` entry; ERP `isActive=false`; new sales task exists.

Assumptions

ERP status-update endpoint is reachable at the time of expiry processing; if not, source indicates the event may remain pending for retry.

Limitations

Not Available beyond retry-pending behavior noted above.

Best Practices

Monitor scheduler job execution/logs to confirm the cron job is running as expected; verify `SUBSCRIPTION_REMINDER_DAYS` is set appropriately for business needs.

Future Improvements

Add automated tests for the expiry and payment webhook paths (per source "Future Improvements", shared with payment module).

References

Source Implementation Analysis §"CRM Scheduler Modules"; §"Complete Business Workflows → Paid Renewal and Expiry Lifecycle"; §"Cron Jobs and Background Tasks".

(End of Volume 1. Continue to Volume 2 for ERP Backend Modules.)